Discriminator Estimates (КППО/ТЗК)

Создано системой Doxygen 1.9.8

1 Главная страница 1
1.1 Дискриминаторные оценки координат (КППО / ТЗК) . . . 1
1.2 </blockquote> . . . 1
1.3 Что это и зачем . . . 1
1.3.1 Аналогия из жизни . . . 1
1.4 Содержание документации . . . 1
1.5 Главная рекомендация (на одну страницу) . . . 2
1.6 Демонстрация работы . . . 2
1.7 Шесть методов одним взглядом . . . 2
1.8 Быстрый старт . . . 3
1.8.1 C: суб-бинная оценка частоты по 5 точкам спектра (рекомендация) . . . 3
1.8.2 C: классический 3-точечный QA на угловой ДН . . . 3
1.8.3 C: автоматический выбор с защитой от монотонной зоны . . . 3
1.9 Структура модуля . . . 3
1.10 Ссылки и дополнительные ресурсы . . . 3
1.10.1 Внутри проекта . . . 3
1.10.2 Markdown-документы (вне Doxygen) . . . 4
1.10.3 Внешние источники . . . 4
1.11 Математические формулы . . . 4
1.11.1 Полное описание формул дискриминаторных оценок . . . 4
1.11.2 1. Центр тяжести (CG — Center of Gravity) . . . 4
1.11.2.1 2-точечный (discr2cg) . . . 4
1.11.2.2 3-точечный (discr3cg) . . . 4
1.11.2.3 N-точечный (discrncg) . . . 4
1.11.3 2. Суммарно-разностный (SD — Sum-Difference) . . . 5
1.11.3.1 2-точечный (discr2sd) . . . 5
1.11.4 3. Квадратичная аппроксимация (QA — Quadratic Approximation) . . . 5
1.11.4.1 3-точечный (discr3qa) . . . 5
1.11.5 4. Экспоненциальная аппроксимация (EA — Exponential Approximation) . . . 5
1.11.5.1 Алгоритм (discr3ea) . . . 5
1.11.5.2 Уточнение амплитуды (discr3eaY) . . . 6
1.11.6 5. МНК-парабола по 5 точкам — 5QA (<tt>discr5qa</tt>) . . . 6
1.11.6.1 Замкнутая формула для равноотстоящих \f$ x_i = \{-2,-1,0,1,2\} \f$. 6
1.11.7 6. МНК-Гауссиан по 5 точкам — 5EA (<tt>discr5ea</tt>) . . . 7
1.11.7.1 Алгоритм . . . 7
1.11.7.2 Когда не работает (и что делать) . . . 7
1.11.7.3 Почему именно Гауссиан . . . 7
1.11.8 7. Сравнение всех методов . . . 7
1.11.8.1 Таблица сравнения . . . 7
1.11.8.2 Вердикт по результатам Монте-Карло (Hanning kernel, zp×2, 105 000 испытаний) . . . 8
1.11.9 8. Обёртка discr3() — коэффициент пересчёта . . . 8
1.12 Визуальный разбор методов . . . 8

1.12.1 Поведение дискриминаторов «вживую» — анимации, 3D-карты и контуры . . . 8
1.12.2 </blockquote> . . . 8
1.12.3 Раздел 1. GIF-анимации — поведение в динамике . . . 8
1.12.3.1 Sweep по всей зоне — все методы вместе . . . 8
1.12.3.2 Крупный план — нормальная зона . . . 9
1.12.3.3 Нарастание шума . . . 9
1.12.3.4 Резкое изменение пика . . . 9
1.12.3.5 Авто-демо всех сценариев . . . 9
1.12.4 Раздел 2. Карты точности — 3D и heatmap . . . 9
1.12.4.1 5EA: точность как функция (x0, SNR) . . . 9
1.12.4.2 Сравнение 4 классических методов на одной картинке . . . 10
1.12.4.3 Тепловые карты по отдельным методам . . . 10
1.12.5 Раздел 3. Чистый сигнал — точность на эталоне . . . 10
1.12.5.1 Все дискриминаторы на sinc(x − 0.25) . . . 10
1.12.5.2 МНК-5 на Hanning kernel . . . 10
1.12.5.3 Ошибка vs смещение пика — sweep . . . 10
1.12.5.4 МНК 5/7/9 точек — без шума . . . 11
1.12.6 Раздел 4. С шумом — устойчивость . . . 11
1.12.6.1 Базовые методы под Монте-Карло шумом . . . 11
1.12.6.2 5pt vs 3pt под шумом — Hanning . . . 11
1.12.6.3 МНК 5 / 7 / 9 точек под шумом . . . 11
1.12.7 Раздел 5. Монотонная зона . . . 11
1.12.8 Куда дальше . . . 11
1.13 Итоговые рекомендации . . . 11
1.13.1 Как найти частоту в зашумлённом сигнале — финальные выводы . . . 11
1.13.2 </blockquote> . . . 11
1.13.3 Главная рекомендация — на одну минуту . . . 11
1.13.3.1 Почему не нужен «гибрид» . . . 12
1.13.4 Главный эксперимент — кто победил . . . 12
1.13.4.1 Левый график — ошибка vs SNR . . . 12
1.13.4.2 Правый график — во сколько раз лучше . . . 12
1.13.5 Что это значит в реальных герцах . . . 12
1.13.5.1 Левый график — при типичных условиях . . . 12
1.13.5.2 Правый график — точность vs SNR . . . 13
1.13.6 Бонус: монотонная зона . . . 13
1.13.6.1 Что это . . . 13
1.13.6.2 Что это значит на практике . . . 13
1.13.7 Почему именно 5 точек, а не 7 или 9 . . . 13
1.13.7.1 Почему так . . . 13
1.13.8 </blockquote> . . . 13
1.13.9 Парабола + Hanning = любовь . . . 13
1.13.9.1 Левый график — форма пика . . . 13
1.13.9.2 Правый график — парабола на Hanning . . . 14

1.13.10 </blockquote> . . . 14
1.13.11 Zero-padding ×2 — обязательное требование . . . 14
1.13.11.1 Что это . . . 14
1.13.11.2 Зачем нам zp×2 минимум . . . 14
1.13.12 Quick reference — как использовать . . . 14
1.13.12.1 C-код (текущий модуль) . . . 14
1.13.12.2 Python-код (для тестов и прототипирования) . . . 15
1.13.13 Чек-лист перед релизом . . . 15
1.13.14 Что дальше — Quinn 2nd и комплексный pipeline . . . 15
1.13.14.1 Quinn 2nd estimator — максимум точности . . . 15
1.13.14.2 IPI (Iterative Parabolic Interpolation, 2025) . . . 16
1.13.14.3 CZT / Zoom FFT . . . 16
1.13.15 Дополнительные ресурсы . . . 16
1.13.16 Самое главное, если коротко . . . 16
1.14 Тестирование . . . 16
1.14.1 4 вида тестов — 77+ тестов . . . 16
1.14.2 1. Простые C++ тесты (<tt>test_cpp/</tt>) — 27 тестов . . . 16
1.14.2.1 Структура . . . 16
1.14.2.2 Паттерн теста . . . 17
1.14.2.3 Результат . . . 17
1.14.3 2. Google Test (<tt>test_gtest/</tt>) — 40 тестов . . . 17
1.14.3.1 Структура . . . 17
1.14.3.2 Ключевые тесты . . . 17
1.14.4 3. Python тесты (<tt>test_python/</tt>) — 7 тестов + 3 графика . . . 17
1.14.4.1 Эталонные реализации . . . 18
1.14.4.2 Результат сравнения точности . . . 18
1.14.4.3 Графики . . . 18
1.14.5 Сборка и запуск . . . 18
1.14.5.1 Простые C++ тесты . . . 18
1.14.5.2 Google Test . . . 18
1.14.5.3 Python . . . 18
1.14.6 4. FFT-дискриминаторы частоты (<tt>test_python/</tt>) — 10 тестов + 2 графика . . . 19
1.14.6.1 Тестовые группы . . . 19
1.14.6.2 Результат . . . 19
1.14.6.3 Графики FFT . . . 19
1.15 FFT-дискриминаторы частоты . . . 19
1.15.1 Оценка частоты по FFT-спектру (из Primer.m) . . . 19
1.15.1.1 Сигнал . . . 19
1.15.2 5 методов оценки частоты . . . 19
1.15.2.1 3-точечные (бины k-1, k, k+1) . . . 19
1.15.2.2 2-точечные (top-2 bins по амплитуде) . . . 20
1.15.3 Дискриминационная характеристика . . . 20

1.15.4 Сравнение точности . . . 20
1.15.5 Влияние оконной функции . . . 20
1.15.6 Запуск тестов . . . 20
1.15.7 Происхождение . . . 20

2 Алфавитный указатель структур данных 21
2.1 Структуры данных . . . 21

3 Список файлов 23
3.1 Файлы . . . 23

4 Структуры данных 25
4.1 Структура ax_pair_t . . . 25
4.1.1 Подробное описание . . . 25
4.1.2 Поля . . . 25
4.1.2.1 a . . . 25
4.1.2.2 x . . . 25
4.2 Структура axstr_t . . . 25
4.2.1 Подробное описание . . . 25
4.2.2 Поля . . . 26
4.2.2.1 a . . . 26
4.2.2.2 x . . . 26

5 Файлы 27
5.1 Файл fft.md . . . 27
5.2 Файл formulas.md . . . 27
5.3 Файл mainpage.md . . . 27
5.4 Файл methods_visual.md . . . 27
5.5 Файл recommendations.md . . . 27
5.6 Файл tests.md . . . 27
5.7 Файл discr5ea.h . . . 27
5.7.1 Подробное описание . . . 27
5.7.2 Функции . . . 28
5.7.2.1 discr5ea() . . . 28
5.7.2.2 discr5qa() . . . 29
5.8 discr5ea.h . . . 30
5.9 Файл discr_auto.h . . . 30
5.9.1 Подробное описание . . . 31
5.9.2 Функции . . . 31
5.9.2.1 discr3_auto() . . . 31
5.9.2.2 discr3_extrap_gauss() . . . 32
5.9.2.3 discr_is_monotonic() . . . 34
5.10 discr_auto.h . . . 35
5.11 Файл discr_common.h . . . 35
5.11.1 Подробное описание . . . 36

5.11.2 Функции . . . 36
5.11.2.1 checkuse3d() . . . 36
5.11.2.2 conv3to2() . . . 37
5.11.2.3 discr2() . . . 38
5.11.2.4 discr3() . . . 39
5.11.2.5 discr3_() . . . 41
5.12 discr_common.h . . . 42
5.13 Файл discr_types.h . . . 43
5.13.1 Подробное описание . . . 43
5.13.2 Перечисления . . . 44
5.13.2.1 DISCRTYPE . . . 44
5.14 discr_types.h . . . 44
5.15 Файл discrcg.h . . . 44
5.15.1 Подробное описание . . . 45
5.15.2 Функции . . . 45
5.15.2.1 discr2cg() . . . 45
5.15.2.2 discr3cg() . . . 46
5.15.2.3 discrncg() . . . 47
5.16 discrcg.h . . . 47
5.17 Файл discrea.h . . . 48
5.17.1 Подробное описание . . . 48
5.17.2 Функции . . . 48
5.17.2.1 discr3ea() . . . 48
5.17.2.2 discr3eaY() . . . 50
5.18 discrea.h . . . 52
5.19 Файл discrqa.h . . . 52
5.19.1 Подробное описание . . . 52
5.19.2 Функции . . . 53
5.19.2.1 discr3qa() . . . 53
5.20 discrqa.h . . . 54
5.21 Файл discrsd.h . . . 54
5.21.1 Подробное описание . . . 55
5.21.2 Функции . . . 55
5.21.2.1 discr2sd() . . . 55
5.22 discrsd.h . . . 56
5.23 Файл discr5ea.c . . . 56
5.23.1 Подробное описание . . . 57
5.23.2 Функции . . . 57
5.23.2.1 discr5ea() . . . 57
5.23.2.2 discr5qa() . . . 58
5.23.2.3 lsq5_peak() . . . 59
5.24 discr5ea.c . . . 60
5.25 Файл discr_auto.c . . . 61

5.25.1 Подробное описание . . . 62
5.25.2 Функции . . . 62
5.25.2.1 cmp_by_x() . . . 62
5.25.2.2 discr3_auto() . . . 62
5.25.2.3 discr3_extrap_gauss() . . . 63
5.25.2.4 discr_is_monotonic() . . . 65
5.26 discr_auto.c . . . 66
5.27 Файл discr_common.c . . . 69
5.27.1 Подробное описание . . . 69
5.27.2 Макросы . . . 70
5.27.2.1 _USE_MATH_DEFINES . . . 70
5.27.3 Функции . . . 70
5.27.3.1 checkuse3d() . . . 70
5.27.3.2 conv3to2() . . . 70
5.27.3.3 discr2() . . . 71
5.27.3.4 discr3() . . . 72
5.27.3.5 discr3_() . . . 74
5.28 discr_common.c . . . 74
5.29 Файл discrcg.c . . . 77
5.29.1 Подробное описание . . . 77
5.29.2 Функции . . . 77
5.29.2.1 discr2cg() . . . 77
5.29.2.2 discr3cg() . . . 78
5.29.2.3 discrncg() . . . 78
5.30 discrcg.c . . . 79
5.31 Файл discrea.c . . . 79
5.31.1 Подробное описание . . . 80
5.31.2 Функции . . . 80
5.31.2.1 compare_by_x() . . . 80
5.31.2.2 discr3ea() . . . 80
5.31.2.3 discr3eaY() . . . 82
5.32 discrea.c . . . 83
5.33 Файл discrqa.c . . . 84
5.33.1 Подробное описание . . . 85
5.33.2 Функции . . . 85
5.33.2.1 discr3qa() . . . 85
5.34 discrqa.c . . . 86
5.35 Файл discrsd.c . . . 86
5.35.1 Подробное описание . . . 87
5.35.2 Функции . . . 87
5.35.2.1 discr2sd() . . . 87
5.36 discrsd.c . . . 88

Предметный указатель 89

Глава 1

Главная страница

1.1 Дискриминаторные оценки координат (КППО / ТЗК)

> Модуль: discriminator_estimates Организация: ОАО НПК НИИДАР Авторы: Федоров С.А., Добродумов А.Б., Керский Е.В. Рефакторинг и расширение: Кодо (AI-ассистент) + Alex, 2026 Версия документации: 2026-04-06

1.2 </blockquote>

1.3 Что это и зачем

Эта библиотека отвечает на один очень практичный вопрос:

> «У меня есть несколько точек измерения сигнала. Где находится истинный максимум — между точками?»

Под «точками» понимаются:

- отсчёты диаграммы направленности (ДН) антенны для оценки угловых координат цели в РЛС;
- бины спектра FFT для оценки точной частоты одиночного гармонического сигнала в зашумлённой записи.

В обоих случаях математика одинаковая, и здесь собраны 6 проверенных методов — от классических 2-точечных до новых 5-точечных МНК-аппроксимаций.

1.3.1 Аналогия из жизни

> Представь, что ты стоишь на стадионе в шумной толпе и пытаешься услышать, на какой ноте поёт один конкретный певец. Толпа — это шум. Голос певца — наш полезный сигнал. У тебя есть три-пять микрофонов, расставленных по частотной шкале — каждый «слышит» немного по-разному. Задача: по этим 3–5 числам угадать, где точно находится голос певца.
>
> То, чем занимается этот модуль — это и есть «найти певца в толпе».

Подробное объяснение для чайника + полная теория есть в файле Doc/Frequency_↩ Recommendations_2026-04-06.md (на этом же документе и основан большой раздел Итоговые рекомендации).

1.4 Содержание документации

#	Раздел	Описание
1	Главная	Назначение, методы, навигация
2	Формулы	Полный математический вывод (CG, SD, QA, EA, 5QA, 5EA)
3	Визуальный разбор	GIF-анимации, 3D-карты, поведение методов
4	Итоговые рекомендации	Что использовать на практике + дерево решений
5	Тестирование	C++ / GTest / Python — 77+ тестов
6	FFT-дискриминаторы	Применение к оценке частоты по спектру

Файлы C API: discrcg.h, discrsd.h, discrqa.h, discrea.h, discr5ea.h, discr_auto.h, discr_common.h, discr_types.h.

1.5 Главная рекомендация (на одну страницу)

Если кратко: для FFT-задачи (поиск частоты) используй **discr5ea с fallback на discr5qa** и обязательным zero-padding ×2 с окном Hanning.

Это даёт ошибку ~0.6 Гц при SNR = 20 дБ для типичного радара (fs = 100 МГц, N = 4M) — в 20 раз лучше, чем без интерполяции, и в 2 раза лучше классического 3-точечного discr3ea.

Подробное обоснование — на странице Итоговые рекомендации.

1.6 Демонстрация работы

Что видно на анимации:

- зелёный пунктир — истинное положение пика (то, что надо найти);
- цветные вертикальные линии — оценки разных методов;
- точки сверху — отсчёты, которые «видит» алгоритм;
- хорошие методы сидят рядом с зелёной линией, плохие — далеко.

Больше анимаций — на странице Визуальный разбор методов.

1.7 Шесть методов одним взглядом

Метод	Файл	Точек	Идея	Краткая формула	Когда хорош
CG	discrcg.h	2…N	Центр тяжести	$x_e = \frac{\sum A_i x_i}{\sum A_i}$	Грубая оценка
SD	discrsd.h	2	Сумма-разность (моноимпульсный)	$x_e = x_c + c \cdot \frac{A_2 - A_1}{A_2 + A_1}$	2 канала, моноимпульс
QA	discrqa.h	3	Парабола по 3 точкам	$x_e = -\frac{b}{2a}$	Параболоподобный пик
EA	discrea.h	3	Парабола в log-масштабе (Гауссиан)	$x_e = \alpha/(2\beta)$	Гауссовый пик
5QA ⋆	discr5ea.h	5	МНК-парабола по 5 точкам	$a = \frac{2y_1 - y_2 - 2y_3 - y_4 + 2y_5}{14}$	Шум, Hanning kernel
5EA ⋆⋆	discr5ea.h	5	МНК-Гауссиан по 5 точкам	то же, но $z_i = \ln y_i$	Универсал, FFT

⋆ — добавлено при рефакторинге 2026, ⋆⋆ — рекомендуемый по умолчанию для FFT.
Полные формулы и вывод — на странице Математические формулы.

1.8 Быстрый старт

1.8.1 C: суб-бинная оценка частоты по 5 точкам спектра (рекомендация)

```c
#include "discr5ea.h"
#include <complex.h>

// X[]    — комплексный спектр после FFT (длина N_padded)
// k_max  — индекс максимума |X(k)| (грубая оценка)
// fs     — частота дискретизации [Гц]
// N_padded — длина FFT с zero-padding ×2

double y[5], xs[5];
for (int i = 0; i < 5; ++i) {
  y[i]  = cabs(X[k_max - 2 + i]);   // |X(k-2)| .. |X(k+2)|
  xs[i] = (double)(i - 2);          // {-2, -1, 0, 1, 2}
}

double delta;
int ok = discr5ea(y, xs, &delta);   // МНК-Гауссиан по 5 точкам
if (ok != 0) {
  // fallback: какая-то |X(k)| ≤ 0 → 5QA по амплитудам
  discr5qa(y, xs, &delta);
}

double f_hz = (k_max + delta) * fs / (double)N_padded;
```

1.8.2 C: классический 3-точечный QA на угловой ДН

```c
#include "discrqa.h"

double A1 = 0.75, A2 = 2.75, A3 = 2.75;
double x1 = -1.0, x2 = 0.0,  x3 = 1.0;
double xe = discr3qa(A1, A2, A3, x1, x2, x3);
// xe ≈ 0.5 — оценка положения пика между точками
```

1.8.3 C: автоматический выбор с защитой от монотонной зоны

```c
#include "discr_auto.h"

double xe;
int how = discr3_auto(A1, A2, A3, x1, x2, x3, &xe);
// how == 0 → EA использован   (нормальный случай)
// how == 1 → QA fallback      (EA не сошёлся)
// how == 2 → E2 экстраполяция (монотонные данные)
// how == 3 → E2 не сошёлся    (xe = x2)
```

1.9 Структура модуля

```
discriminator_estimates/
+- include/              ← публичный C API (8 заголовков)
|   +- discrcg.h        ← Центр тяжести
|   +- discrsd.h        ← Сумма-разность
|   +- discrqa.h        ← Парабола (3pt)
|   +- discrea.h        ← Гауссиан (3pt)
|   +- discr5ea.h       ← МНК 5pt: 5QA + 5EA  ⋆ новое
|   +- discr_auto.h       ← Авто-выбор + extrap ⋆ новое
|   +- discr_common.h       ← Обёртки discr2/discr3
|   +- discr_types.h      ← DISCRTYPE enum
+- src/                  ← реализация (.c)
+- test_cpp/              ← простые C++ тесты
+- test_gtest/            ← Google Test (40+ тестов)
+- test_python/            ← Python тесты + графики
+- Doc/
   +- API.md  Full.md  Quick.md   ← markdown-документация
   +- Doxygen/            ← вы здесь
   +- plots/             ← все графики и анимации
   |   +- 1_animation/     ← .gif
   |   +- 2_no_noise/      ← чистый сигнал
   |   +- 3_noise/         ← Монте-Карло с шумом
   |   +- 4_3d/            ← 3D-карты, heatmap
   |   +- 5_recommendations/ ← итоговые иллюстрации
   +- Review/              ← история рефакторинга
```

1.10 Ссылки и дополнительные ресурсы

1.10.1 Внутри проекта

- Математические формулы — Полные формулы всех 6 методов (CG, SD, QA, EA, 5QA, 5EA)

- Визуальный разбор методов — Визуальный разбор: GIF, 3D-карты, поведение
- Итоговые рекомендации — Итоговые рекомендации + дерево решений
- Тестирование — Тестирование (C++ / GTest / Python — 77+ тестов)
- FFT-дискриминаторы частоты — FFT-дискриминаторы частоты

1.10.2 Markdown-документы (вне Doxygen)

Эти файлы лежат рядом с Doxygen-документацией, в каталоге Doc/:

- Doc/API.md — справочник публичного API
- Doc/Full.md — полная документация модуля
- Doc/Quick.md — быстрый старт
- Doc/Frequency_Recommendations_2026-04-06.md — итоговый отчёт «как найти частоту в шуме» (на котором построена Итоговые рекомендации)
- Doc/FFT_Frequency_Estimation.md — справочник по FFT-методам суб-бинной оценки
- Doc/plots/README.md — описание всех графиков с пояснениями

1.10.3 Внешние источники

1. Gasior, Gonzalez (CERN, 2004) — Improving FFT frequency measurement resolution by parabolic and Gaussian interpolation
2. Julius O. Smith III — Quadratic Interpolation of Spectral Peaks
3. Quinn, B.G. (IEEE TSP, 1994, 1997) — оригинальные статьи по Quinn estimators
4. Candan, Ç. (2011) — Fine resolution frequency estimation from three DFT samples

---

1.11 Математические формулы

1.11.1 Полное описание формул дискриминаторных оценок

1.11.2 1. Центр тяжести (CG — Center of Gravity)

Простейший метод. Координата оценивается как взвешенное среднее.

1.11.2.1 2-точечный (discr2cg)

$$x_e = \frac{A_1 \cdot x_1 + A_2 \cdot x_2}{A_1 + A_2}$$

Геометрический смысл: координата центра масс двух точек с "массами" A_1 и A_2.

1.11.2.2 3-точечный (discr3cg)

$$x_e = \frac{A_1 \cdot x_1 + A_2 \cdot x_2 + A_3 \cdot x_3}{A_1 + A_2 + A_3}$$

1.11.2.3 N-точечный (discrncg)

$$x_e = \frac{\sum_{i=0}^{N-1} A_i \cdot x_i}{\sum_{i=0}^{N-1} A_i}$$

Защита: при $\sum A_i = 0$ возвращается средняя точка (ERR-005 fix).

1.11.3 2. Суммарно-разностный (SD — Sum-Difference)

Классический моноимпульсный метод. Использует отношение разности к сумме.

1.11.3.1 2-точечный (discr2sd)

$$x_c = \frac{x_1 + x_2}{2}$$

$$\Delta x = c \cdot \frac{A_2 - A_1}{A_2 + A_1}$$

$$x_e = x_c + \Delta x$$

где c — коэффициент крутизны дискриминаторной характеристики.
Физический смысл: $\frac{A_2 - A_1}{A_2 + A_1}$ — нормированная разность, пропорциональная отклонению цели от середины. Коэффициент c зависит от ширины ДН и шага сетки.

1.11.4 3. Квадратичная аппроксимация (QA — Quadratic Approximation)

Через 3 точки проводится парабола $y = ax^2 + bx + c$, находится вершина.

1.11.4.1 3-точечный (discr3qa)

$$A_o = \frac{A_2 - A_1}{A_2 - A_3}$$

$$x_e = \frac{1}{2} \cdot \frac{(A_o - 1) \cdot x_2^2 - A_o \cdot x_3^2 + x_1^2}{(A_o - 1) \cdot x_2 - A_o \cdot x_3 + x_1}$$

Граничные случаи (ERR-004 fix, используется DBL_EPSILON):

Условие	Результат
$A_1 = A_2 = A_3$	$x_e = x_2$ (центральная точка)
$A_2 = A_3$, $A_1 > A_2$	$x_e = x_1$
$A_2 = A_3$, $A_1 < A_2$	$x_e = \frac{x_2 + x_3}{2}$
$A_1 = A_2$, $A_3 > A_2$	$x_e = x_3$
$A_1 = A_2$, $A_3 < A_2$	$x_e = \frac{x_1 + x_2}{2}$

1.11.5 4. Экспоненциальная аппроксимация (EA — Exponential Approximation)

Наиболее точный метод. Моделирует ДН функцией Гаусса:

$$y(x) = A_{max} \cdot \exp\left(-a \cdot (x - x_0)^2\right)$$

1.11.5.1 Алгоритм (discr3ea)

Шаг 1: Проверка входных данных

- Все амплитуды должны быть > 0
- Не все амплитуды равны

Шаг 2: Сортировка по возрастанию координаты x
Шаг 3: Проверка выпуклости

- Максимум должен быть в центре (а не на краю)
- При вогнутости $\rightarrow$ EXIT_FAILURE

Шаг 4: Логарифмирование

$$z_i = \ln(A_i), \quad i = 1, 2, 3$$

Задача сводится к параболической аппроксимации в логарифмическом масштабе: $z(x) = -a(x - x_0)^2 + \ln(A_{max})$
Шаг 5: Вершина параболы

$$\alpha = z_1(f_2^2 - f_3^2) + z_2(f_3^2 - f_1^2) + z_3(f_1^2 - f_2^2)$$

$$\beta = z_1(f_2 - f_3) + z_2(f_3 - f_1) + z_3(f_1 - f_2)$$

$$x_e = \frac{\alpha}{2\beta}$$

где f_i — координаты после сортировки.
Шаг 6: Ограничение на вылет

$$f_1 - \frac{f_3 - f_1}{2} \le x_e \le f_3 + \frac{f_3 - f_1}{2}$$

Оценка не должна выходить за пределы области зондирования более чем на полшага.

1.11.5.2 Уточнение амплитуды (discr3eaY)

$$a_0 = \frac{z_1 - z_2}{2x_e(x_1 - x_2) - x_1^2 + x_2^2}$$

$$y_e = A_2 \cdot \exp\left(a_0 \cdot (x_2 - x_e)^2\right)$$

1.11.6 5. МНК-парабола по 5 точкам — 5QA (<tt>discr5qa</tt>)

Идея: вместо параболы, идеально проходящей через 3 точки, аппроксимируем параболой 5 точек по методу наименьших квадратов (МНК / LSQ). Получаем переопределённую систему — 5 уравнений на 3 неизвестных (a, b, c):

$$A_i \approx a\, x_i^2 + b\, x_i + c, \qquad i = 1, \ldots, 5$$

В матричной форме:

$$\mathbf{H}\,\boldsymbol{\theta} = \mathbf{y}, \quad \mathbf{H} = \begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \\ x_4^2 & x_4 & 1 \\ x_5^2 & x_5 & 1 \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}.$$

Решение МНК (нормальные уравнения):

$$(\mathbf{H}^\top \mathbf{H})\,\boldsymbol{\theta} = \mathbf{H}^\top \mathbf{y}.$$

1.11.6.1 Замкнутая формула для равноотстоящих \f$ x_i = \{-2,-1,0,1,2\} \f$

Для равномерной сетки $x_i = -2, -1, 0, 1, 2$ матрица $\mathbf{H}^\top \mathbf{H}$ блочно-диагональна (так как сумма нечётных степеней равна нулю), и решение можно записать в замкнутом виде:

$$\boxed{a = \frac{2A_1 - A_2 - 2A_3 - A_4 + 2A_5}{14}}$$

$$\boxed{b = \frac{-2A_1 - A_2 + A_4 + 2A_5}{10}}$$

$$\boxed{\delta = -\frac{b}{2a}, \qquad x_e = x_3 + h\,\delta}$$

где h — шаг сетки, x_3 — центральная точка.
Свойства:

- Сложность: O(1) — 6 умножений + 7 сложений + 1 деление
- Не требует никаких циклов и матричных вычислений
- В нормальной зоне даёт ошибку ~2× меньше, чем 3-точечная QA

Заметки

Защита: при $|a| < 10^{-30}$ возвращается $x_e = x_3$ (данные плоские — оценить нельзя).

1.11.7 6. МНК-Гауссиан по 5 точкам — 5EA (<tt>discr5ea</tt>)

Самый точный из реализованных методов. Аппроксимирует пик гауссианом (колоколом) методом наименьших квадратов:

$$y(x) \approx A_{\max} \cdot \exp\!\left(-a\,(x - x_0)^2\right)$$

1.11.7.1 Алгоритм

Шаг 1. Логарифмирование амплитуд:

$$z_i = \ln A_i, \quad i = 1, \ldots, 5$$

В лог-пространстве гауссиан превращается в параболу:

$$z(x) = -a\,(x - x_0)^2 + \ln A_{\max} = a'x^2 + b'x + c'.$$

Шаг 2. К z[] применяем те же замкнутые МНК-формулы, что и в 5QA:

$$a = \frac{2z_1 - z_2 - 2z_3 - z_4 + 2z_5}{14}$$

$$b = \frac{-2z_1 - z_2 + z_4 + 2z_5}{10}$$

Шаг 3. Вершина параболы → оценка x_e:

$$\delta = -\frac{b}{2a}, \qquad x_e = x_3 + h\,\delta.$$

1.11.7.2 Когда не работает (и что делать)

Метод требует строго положительных амплитуд: $A_i > 0$. Иначе $\ln A_i$ не определён.
Стратегия fallback в боевом коде:

```
int ok = discr5ea(A, x, &xe);
if (ok != 0) {
  // что-то ≤ 0 или результат за границей
  discr5qa(A, x, &xe);   // ← переходим на 5QA по амплитудам
}
```

1.11.7.3 Почему именно Гауссиан

Главное наблюдение проекта: после применения окна Hanning и FFT форма одиночного спектрального пика (Hanning kernel) очень близка к гауссиану в пределах главного лепестка. Поэтому модель $y \approx A_{\max} \exp(-a(x - x_0)^2)$ является физически обоснованной для типичного FFT-пайплайна.

1.11.8 7. Сравнение всех методов

1.11.8.1 Таблица сравнения

Метод	Точек	Bias (без шума)	Шумоустойчивость	Сложность	Требования
CG	2…N	~0.17 шага	низкая	O(N)	$\sum A_i \neq 0$
SD	2	~0.10 шага	средняя	O(1)	подобрать c
QA	3	~0.01 шага	средняя	O(1)	A_i любые
EA	3	~0.005 шага	средняя	O(1)	$A_i > 0$
5QA	5	~0.005 шага	высокая	O(1)	A_i любые
5EA ⋆	5	~0.003 шага	высокая	O(1)	$A_i > 0$

1.11.8.2 Вердикт по результатам Монте-Карло (Hanning kernel, zp×2, 105 000 испытаний)

SNR (дБ)	Выигрыш лучшего 5pt относительно EA3
40 (идеал)	1.4×
30 (отлично)	1.8×
25 (хорошо)	2.0×
20 (норма)	2.0×
15 (шумно)	1.7×
10 (плохо)	1.2×
5 (ужас)	1.2×

5pt-методы стабильно выигрывают в 1.2–2.0 раза при любом уровне шума. Подробное обсуждение и дерево выбора — на странице Итоговые рекомендации.

1.11.9 8. Обёртка discr3() — коэффициент пересчёта

Для обёрток discr3() и discr3_() координаты пересчитываются:

$$c = \frac{2\pi \cdot dx}{\lambda}$$

где dx — шаг антенной решётки, λ — длина волны.
Координаты масштабируются: $x' = x \cdot c$, оценка возвращается в исходных единицах: $x_e = x'_e/c$.

Заметки

После рефакторинга (ERR-003 fix) входные массивы не модифицируются — координаты умножаются на c при передаче в дискриминатор, а не в массиве.

1.12 Визуальный разбор методов

1.12.1 Поведение дискриминаторов «вживую» — анимации, 3D-карты и контуры

Самый быстрый способ понять, как ведут себя методы — посмотреть, как они реагируют на движущийся пик и нарастающий шум. На этой странице собраны все наглядные иллюстрации модуля.

1.12.2 </blockquote>

1.12.3 Раздел 1. GIF-анимации — поведение в динамике

1.12.3.1 Sweep по всей зоне — все методы вместе

Что происходит на кадре:

- по горизонтали — координата;
- голубая кривая — sinc/Hanning kernel с движущимся пиком;

- зелёный пунктир — истинное положение пика x_0;
- цветные вертикальные линии — оценки разных методов;
- крайние точки слева/справа — отсчёты, которые «видит» алгоритм.

На что смотреть:

- В центральной зоне ($|x_0| \leq 0.5$ шага) — все методы вместе.
- Ближе к краям — 3-точечные методы (CG, SD, QA, EA) разбегаются: пик уходит из их «зоны видимости».
- 5-точечные (5QA, 5EA) держат пик намного дольше и не отрываются от зелёной пунктирной линии.

1.12.3.2 Крупный план — нормальная зона

То же самое, но крупным планом, в пределах нормальной зоны $|x_0| \leq 1$ шаг. Здесь видно даже мелкие различия между методами:

- EA сидит почти на самой зелёной линии,
- QA немного дрожит относительно неё,
- CG/SD ощутимо смещены.

1.12.3.3 Нарастание шума

Что происходит: пик стоит на месте, а шум плавно нарастает (SNR с 40 дБ падает до 0 дБ). Каждый кадр — новая случайная реализация шума.
На что смотреть:

- При высоком SNR все оценки сидят рядом с истинным пиком.
- При низком SNR оценки начинают «дрожать». Чем стабильнее метод — тем меньше амплитуда дрожания.
- 5EA / 5QA дрожат заметно меньше, чем 3-точечные собратья — это визуальное подтверждение их шумоустойчивости из Итоговые рекомендации.

1.12.3.4 Резкое изменение пика

Тестируем переходный процесс: пик мгновенно скачет с одной координаты на другую. Никакого сглаживания между кадрами — каждый кадр это новый независимый замер.
Что показывает: все методы немедленно реагируют (нет инерции — дискриминаторы не имеют памяти). Полезно как контроль того, что в реализации нет случайной фильтрации.

1.12.3.5 Авто-демо всех сценариев

Все три сценария подряд в одном GIF-файле — удобно для презентаций. Подробное объяснение каждого фрагмента — в трёх разделах выше.

1.12.4 Раздел 2. Карты точности — 3D и heatmap

1.12.4.1 5EA: точность как функция (x0, SNR)

Это самая важная карта — показывает, как ведёт себя рекомендуемый метод 5EA в двумерном пространстве ключевых параметров: смещение пика x_0 и уровень шума SNR.
Что видно:

- Левая нижняя зона (малый x_0, высокий SNR) — тёмная = маленькая ошибка. Это идеальные условия: пик по центру окна, шум слабый.

- Правая зона (большой x_0) — резкий рост ошибки. Это монотонная зона (пик вышел за пределы 3-точечного окна).
- Верхняя зона (низкий SNR) — ошибка растёт при любом x_0: шум слишком сильный.
- Голубая линия на 3D-поверхности — граница монотонной зоны для 3pt методов. Правее этой линии 5EA продолжает работать, а QA(3pt) уже сыплется.

Практический вывод: чтобы получить ошибку < 0.05 шага, нужно одновременно:

- удерживать SNR > 15 дБ И
- держать ** $|x_0| < 1.5$ шага** от центра окна.

1.12.4.2 Сравнение 4 классических методов на одной картинке

Четыре 3D-поверхности рядом. Сравниваем — у какого метода «долина» глубже (меньше ошибка) и шире (больше допустимый диапазон x_0).
Типичный вывод:

- EA — самая глубокая долина (точнее всех на чистом сигнале);
- CG — самая широкая, но мелкая (грубо, но устойчиво);
- QA — промежуточный вариант;
- SD — нестабилен при неправильном выборе коэффициента c.

1.12.4.3 Тепловые карты по отдельным методам

CG	SD	QA	EA

Тёмная горизонтальная полоса = оптимальный диапазон шага сетки d. Именно эту полосу нужно искать, выбирая шаг под свою задачу.

1.12.5 Раздел 3. Чистый сигнал — точность на эталоне

1.12.5.1 Все дискриминаторы на sinc(x − 0.25)

Что нарисовано:

- Голубая линия — sinc(x − 0.25), реалистичная ДН антенны;
- Жёлтые точки — 3 отсчёта на сетке $\{-1, 0, +1\}$;
- Вертикальные линии — оценки CG / QA / EA / 5EA;
- Зелёная пунктирная — истинное положение пика $x_0 = 0.25$.

EA и 5EA сидят почти на самой зелёной линии. CG ощутимо левее.

1.12.5.2 МНК-5 на Hanning kernel

Тот же эксперимент, но на Hanning kernel (форма пика после FFT с Hanning-окном). 5 точек, две оценки 5QA / 5EA, обе очень близки к истинному значению.
Это визуальное подтверждение тезиса «парабола + Hanning = любовь» из Итоговые рекомендации.

1.12.5.3 Ошибка vs смещение пика — sweep

Базовый бенчмарк. По горизонтали — истинное смещение пика x_0, по вертикали — модуль ошибки. Чем ниже кривая, тем точнее метод.
Наблюдения:

- CG и SD — почти линейно растут (классическая систематическая ошибка взвешенного среднего);
- QA — почти 0 в нормальной зоне, потом резкий рост;
- EA — самая низкая в нормальной зоне.

1.12.5.4 МНК 5/7/9 точек — без шума

Главное наблюдение: EA-кривые в ∼10× ниже QA-кривых при любом числе точек. Логарифмирование амплитуд (т.е. подгонка гауссиана вместо параболы) действительно даёт большой выигрыш на Hanning kernel.

1.12.6 Раздел 4. С шумом — устойчивость

1.12.6.1 Базовые методы под Монте-Карло шумом

При снижении SNR оценки разлетаются. Метод с меньшим СКО = более стабильный.

1.12.6.2 5pt vs 3pt под шумом — Hanning

5pt-методы выигрывают всегда — а особенно при SNR < 20 дБ.

1.12.6.3 МНК 5 / 7 / 9 точек под шумом

При SNR > 20 дБ — все примерно одинаково. При SNR < 15 дБ — 7pt и 9pt заметно лучше. Но это редкие условия — для типичного радара 5pt оптимум, см. Итоговые рекомендации.

1.12.7 Раздел 5. Монотонная зона

Главный вывод по серии: в монотонной зоне (пик за пределами окна) 3-точечные методы сильно ошибаются, а 5-точечные продолжают работать. Подробнее — раздел «Монотонная зона» на странице Итоговые рекомендации.

1.12.8 Куда дальше

- Математические формулы — Все формулы CG / SD / QA / EA / 5QA / 5EA
- Итоговые рекомендации — Итоговые рекомендации + дерево решений
- Тестирование — Описание тестового покрытия
- FFT-дискриминаторы частоты — Применение к оценке частоты по спектру FFT
- Doc/plots/README.md — полный гайд по всем графикам с пояснениями

1.13 Итоговые рекомендации

1.13.1 Как найти частоту в зашумлённом сигнале — финальные выводы

> Дата: 2026-04-06 Источник: Doc/Frequency_Recommendations_2026-04-06.md (полная версия) Что здесь: концентрат рекомендаций на основе всех проведённых Монте-Карло экспериментов и аналитики.

1.13.2 </blockquote>

1.13.3 Главная рекомендация — на одну минуту

```
+———————————————————————+
|                                |
|  ВСЕГДА используй  discr5ea          |
|  (МНК-Гауссиан по 5 точкам)          |
|                                |
|  Fallback: если какая-то |X(k)| ≤ 0        |
|          → переключись на discr5qa        |
|                                |
```

```
|  Обязательные требования:                    |
|    • Окно Hanning перед FFT                   |
|    • Zero-padding минимум ×2                  |
|    • 5 точек вокруг максимума (k-2 .. k+2)      |
|                                              |
+———————————————————————+
```

1.13.3.1 Почему не нужен «гибрид»

Раньше думали: «при шуме один метод, при чистоте другой». На Hanning kernel гибрид не нужен:

- 5EA и 5QA дают практически одинаковый результат (разница ∼5%);
- Оба стабильно лучше любого 3-точечного метода;
- 5EA чуть точнее на чистых данных, 5QA чуть точнее при сильном шуме — но разница исчезающая.

Один вызов. Никаких if-else по уровню шума.

1.13.4 Главный эксперимент — кто победил

Эксперимент: для каждого уровня SNR — 5000 × 21 = 105 000 испытаний, случайный шум, случайное положение пика, окно Hanning, zp×2.

1.13.4.1 Левый график — ошибка vs SNR

- Серая пунктирная — EA3 (классика, 3 точки)
- Оранжевая — 5QA (МНК-парабола, 5 точек)
- Синяя жирная ⋆ — 5EA (МНК-Гауссиан, 5 точек)

Главное наблюдение: обе 5-точечные кривые лежат ниже 3-точечной EA3 при любом SNR. То есть 5-точечные всегда лучше.

1.13.4.2 Правый график — во сколько раз лучше

SNR (дБ)	Выигрыш 5pt относительно EA3
40 (идеал)	1.4×
30 (отлично)	1.8×
25 (хорошо)	2.0×
20 (норма)	2.0×
15 (шумно)	1.7×
10 (плохо)	1.2×
5 (ужас)	1.2×

5-точечные методы стабильно выигрывают в 1.2–2.0 раза.

1.13.5 Что это значит в реальных герцах

1.13.5.1 Левый график — при типичных условиях

Параметры: fs = 100 МГц, N = 4M, $\Delta f \approx 23.84$ Гц, SNR = 20 дБ.

Метод	Ошибка
Без интерполяции (просто argmax)	**∼12 Гц**
EA3 (классика, 3 точки)	**∼1.2 Гц**
5EA / 5QA ⋆ (наш выбор)	**∼0.6 Гц**
Quinn 2nd (будущее, по комплексным X(k))	**∼2.4 мГц**

Переход «argmax → 5EA» = улучшение в 20 раз без единого нового датчика — только за счёт правильной математики.

1.13.5.2 Правый график — точность vs SNR

Как ошибка 5EA/5QA в герцах меняется при разном SNR:

- SNR = 30 дБ → ошибка ≈ 0.21 Гц
- SNR = 20 дБ → ошибка ≈ 0.61 Гц
- SNR = 10 дБ → ошибка ≈ 1.73 Гц

Даже в очень шумных условиях (SNR = 10 дБ) мы в 7 раз точнее, чем без интерполяции.

1.13.6 Бонус: монотонная зона

1.13.6.1 Что это

«Монотонная зона» — ситуация, когда истинный пик вышел за пределы окна, и в окне измерений амплитуды только растут или только убывают.
| Метод | Граница монотонной зоны | |----—|----------—| | 3pt (QA, EA) | $|x_0| > 0.5$ бина | | 5pt (5QA, 5EA) | ** $|x_0| > 2.0$ бина** |
5-точечные методы держат пик в 4 раза более широкой зоне.

1.13.6.2 Что это значит на практике

После грубого argmax оценка k_max может промахнуться на половину бина. 5pt прощает промах до 2 бинов — он «дотягивает» правильное значение. То есть 5pt робастнее к небольшим сдвигам пика.

1.13.7 Почему именно 5 точек, а не 7 или 9

Логичный вопрос: «Если 5 лучше 3, может 7 или 9 ещё лучше?»
Результат Монте-Карло: 7 и 9 точек почти не дают выигрыша на Hanning kernel. А при высоком SNR ($\geq$ 25 дБ) 7 и 9 даже ХУЖЕ, чем 5.

1.13.7.1 Почему так

Hanning-пик — это колокол шириной ±2 бина. Крайние точки 7pt/9pt окон (±3, ±4 бина) попадают в хвосты, где:

- форма уже не параболическая (большой bias);
- амплитуды маленькие (плохое отношение сигнал/шум).

Классический trade-off:

- 3 точки — мало данных, шум не усредняется, но bias минимальный;
- 5 точек — золотая середина, шум усредняется, bias ещё мал;
- 7/9 точек — шум чуть лучше усреднён, но bias растёт сильно.

  Вывод: для Hanning kernel 5 точек = оптимум. Больше не нужно.

1.13.8 </blockquote>

1.13.9 Парабола + Hanning = любовь

1.13.9.1 Левый график — форма пика

- Красная (sinc) — форма пика без окна. Острый, с «ушами» (боковыми лепестками −13 дБ).
- Синяя (Hanning kernel) — форма пика с окном Hanning. Гладкий, колоколообразный, без уродливых «ушей».

1.13.9.2 Правый график — парабола на Hanning

5 точек на Hanning-пике + парабола $y = ax^2 + bx + c \rightarrow$ ошибка меньше 2.5% в пределах ±0.7 бина от вершины.

> Аналогия: гора в тумане. У тебя 5 альтиметров, расставленных через равные промежутки. По их показаниям ты хочешь угадать, где самая высокая точка горы между приборами. Если форма горы — парабола (а Hanning kernel очень похож!), достаточно подогнать параболу к 5 точкам и взять её вершину. Это и есть наш метод.

1.13.10 </blockquote>

1.13.11 Zero-padding ×2 — обязательное требование

1.13.11.1 Что это

Перед FFT дописываем нули к сигналу, чтобы общая длина стала в 2 раза больше. Сетка бинов FFT становится гуще, а форма Hanning kernel не меняется.

Вариант	Шаг бинов	Макс. ошибка argmax
Без zero-padding	1.0 бин	±0.5 бина
Zero-pad ×2 ⋆	0.5 бина	**±**0.25 бина
Zero-pad ×4	0.25 бина	±0.125 бина

1.13.11.2 Зачем нам zp×2 минимум

При zp×2 все 5 точек 5EA/5QA попадают в главный лепесток Hanning — там, где парабола ложится идеально. При zp×1 крайние точки вылезают из главного лепестка → парабола больше не работает хорошо.
Экспериментальное подтверждение:

- без zero-padding: ошибка 5QA = 11.9%
- с zp×2: ошибка 5QA = 2.9%

Разница в 4 раза.

Заметки

> Zero-padding не улучшает разрешение — два очень близких пика от него не станут различимыми. Разрешение определяется длиной оригинальной записи.

1.13.12 Quick reference — как использовать

1.13.12.1 C-код (текущий модуль)

```
#include "discr5ea.h"
#include "discr_common.h"
#include <complex.h>
#include <stdlib.h>

// ... после FFT и поиска k_max ...

double y[5], xs[5];
for (int i = 0; i < 5; ++i) {
  y[i]  = cabs(X[k_max - 2 + i]);   // |X(k-2)| .. |X(k+2)|
  xs[i] = (double)(i - 2);          // {-2, -1, 0, 1, 2}
}

double delta;   // суб-бинное смещение в единицах padded-шага
int ok = discr5ea(y, xs, &delta);   // МНК-Гауссиан по 5 точкам

if (ok != EXIT_SUCCESS) {
  // fallback: какая-то амплитуда ≤ 0 → 5QA по амплитудам
  discr5qa(y, xs, &delta);
}

// Перевод в частоту
double f_hz = (k_max + delta) * fs / (double)N_padded;
```

1.13.12.2 Python-код (для тестов и прототипирования)

```
import numpy as np
from scipy.signal.windows import hann

def estimate_frequency(signal, fs, zero_pad=2, search_radius=1000):
    M = len(signal)
    N_base = 1 « (M - 1).bit_length()      # до ближайшей степени 2
    N = N_base * zero_pad

    X = np.fft.fft(signal * hann(M), n=N)
    X_mag = np.abs(X[:N // 2])

    k_max = int(np.argmax(X_mag[:search_radius]))

    if 2 <= k_max <= len(X_mag) - 3:
        y = X_mag[k_max - 2 : k_max + 3]
        if np.all(y > 0):
            z = np.log(y)                  # 5EA
            a = (2*z[0] - z[1] - 2*z[2] - z[3] + 2*z[4]) / 14.0
            b = (-2*z[0] - z[1] + z[3] + 2*z[4]) / 10.0
        else:                              # 5QA fallback
            a = (2*y[0] - y[1] - 2*y[2] - y[3] + 2*y[4]) / 14.0
            b = (-2*y[0] - y[1] + y[3] + 2*y[4]) / 10.0
        delta = -b / (2*a) if abs(a) > 1e-30 else 0.0
    else:
        delta = 0.0

    return (k_max + delta) * fs / N
```

1.13.13 Чек-лист перед релизом

- [ ] Сигнал умножен на окно Hanning до FFT
- [ ] Используется zero-padding минимум ×2 (N = next_pow2(M) ∗ 2)
- [ ] Грубый максимум k_max ищется в разумном диапазоне (не на краях FFT)
- [ ] Вокруг максимума есть минимум 2 точки слева и 2 справа
- [ ] Для discr5ea: проверка A_i > 0 перед вызовом, иначе fallback на discr5qa
- [ ] Результат переводится в Гц через (k_max + δ) · fs / N_padded
- [ ] Покрытие тестами: см. Тестирование
- [ ] Проверена точность на эталонных данных (sinc, Hanning kernel, реальные FFT)

1.13.14 Что дальше — Quinn 2nd и комплексный pipeline

Наш текущий pipeline работает с магнитудой |X(k)| — мы выбрасываем фазу. Но фаза содержит половину информации о сигнале.

1.13.14.1 Quinn 2nd estimator — максимум точности

Если в будущем pipeline сможет передавать комплексные X(k) (а не только магнитуду), можно использовать Quinn 2nd estimator:

Свойство	Значение
Точность	достигает Cramér-Rao bound (теоретический предел)
Точек	3 бина (как у EA3)
Bias	±0.0001 бина (в 50× точнее нашего 5EA)
Сложность	∼20 операций

Это путь для модулей, где комплексные данные есть «из коробки».

1.13.14.2 IPI (Iterative Parabolic Interpolation, 2025)

Свежий итеративный метод. Достигает CRLB, но каждая итерация = O(N). Для 1.3M точек это дорого. Для CPU pipeline неактуально.

1.13.14.3 CZT / Zoom FFT

Альтернатива FFT: вычисляет спектр только в узкой полосе вокруг пика с любым разрешением. Если нужна экстремальная точность (сотни раз лучше нашего 5EA) и не жалко O(N log N) — это вариант.

1.13.15 Дополнительные ресурсы

- Полный отчёт (с историей, аналогиями, словарём терминов): Doc/Frequency_Recommendations↩_2026-04-06.md
- Справочник FFT-методов: Doc/FFT_Frequency_Estimation.md
- Детальный отчёт по Монте-Карло (5pt): Doc/Review/2026-04-03_5pt_LSQ_comparison.md
- Все формулы методов: Математические формулы
- Визуальный разбор: Визуальный разбор методов

1.13.16 Самое главное, если коротко

Для задачи «найти частоту в зашумлённом сигнале» в текущем CPU-pipeline используй discr5ea (МНК-Гауссиан по 5 точкам) с fallback на discr5qa и обязательным zero-padding ×2.

Это даст точность ~0.6 Гц при SNR = 20 дБ — в 20 раз лучше, чем без интерполяции, и в 2 раза лучше, чем классическая 3-точечная EA3.

В будущем (когда появится доступ к комплексным X(k)) — перейти на Quinn 2nd для максимальной точности в ±2 мГц.

1.14 Тестирование

Связанные страницы: Математические формулы (формулы методов), Визуальный разбор методов (визуальный разбор), Итоговые рекомендации (итоговые рекомендации).

1.14.1 4 вида тестов — 77+ тестов

Все тесты используют экспериментальные данные, моделирующие ДН антенны:

$$\mathrm{sinc}(x - x_0) = \frac{\sin(x - x_0)}{x - x_0}$$

где x_0 — истинное смещение пика (цели) относительно центра сетки.

1.14.2 1. Простые C++ тесты (<tt>test_cpp/</tt>) — 27 тестов

Фреймворк: без фреймворка, assert + std::cout Запуск: ./build/test_discr
Быстрые тесты для разработчиков C/C++. Не требуют внешних зависимостей.

1.14.2.1 Структура

Файл	Тестов	Что проверяет
test_discr_cg.hpp	7	CG: 2/3/N-точечный, деление на 0
test_discr_sd.hpp	3	SD: базовый, смещение, деление на 0
test_discr_qa.hpp	5	QA: парабола, граничные A1=A2, A2=A3

Файл	Тестов	Что проверяет
test_discr_ea.hpp	6	EA: гауссиан, вогнутость, нулевые ампл.
test_discr_common.hpp	6	Обёртки: discr2, discr3_, conv3to2

1.14.2.2 Паттерн теста

```
extern "C" {
#include "discrcg.h"
}
#include <cassert>

void test_symmetric() {
  double r = discr3cg(1.0, 3.0, 1.0, -1.0, 0.0, 1.0);
  assert(fabs(r - 0.0) < 1e-12);
  std::cout « "[PASS] discr3cg: symmetric -> center\n";
}
```

1.14.2.3 Результат

```
=== CG Tests ===
[PASS] discr2cg: equal amplitudes -> midpoint
[PASS] discr2cg: dominant A2 -> near x2
[PASS] discr2cg: zero amplitudes -> midpoint (ERR-005)
...
==========================================
ALL DISCRIMINATOR TESTS COMPLETE (27/27)
==========================================
```

1.14.3 2. Google Test (<tt>test_gtest/</tt>) — 40 тестов

Фреймворк: Google Test v1.15.2 (CMake FetchContent) Запуск: ./build/test_discr_gtest или ctest --test-dir build
Полные тесты с подробной диагностикой. Используют sinc(x) данные — реалистичные отсчёты ДН антенны с разными смещениями пика.

1.14.3.1 Структура

Файл	Тестов	Данные
test_discr_cg_gtest.cpp	11	sinc(x), sinc(x-0.3), 5 точек
test_discr_sd_gtest.cpp	5	sinc(x), sinc(x-0.2)
test_discr_qa_gtest.cpp	8	sinc(x-0.2), sinc(x-0.4) мелкая сетка
test_discr_ea_gtest.cpp	9	гауссиан, sinc(x-0.2), вогнутость
test_discr_common_gtest.cpp	7	sinc(x)∗sinc(y) 3x3, ERR-003 побочные эффекты

1.14.3.2 Ключевые тесты

Проверка точности на sinc(x-0.2):

```
TEST(Discr3QA, SincData_Peak_at_02) {
  double x0 = 0.2;
  double A1 = sinc(-1.0 - x0);
  double A2 = sinc(0.0 - x0);
  double A3 = sinc(1.0 - x0);
  double r = discr3qa(A1, A2, A3, -1.0, 0.0, 1.0);
  EXPECT_NEAR(r, x0, 0.15);  // QA: ±0.1 шага
}
```

Проверка ERR-003 (нет побочных эффектов):

```
TEST(Discr3, QA_NoSideEffect_ERR003) {
  double maz_before = maz[0][0];
  discr3_(maz, mel, mval, DT_QA, 0.1, 0.03, &az, &el, &val);
  EXPECT_DOUBLE_EQ(maz[0][0], maz_before);  // входные данные НЕ изменены
}
```

1.14.4 3. Python тесты (<tt>test_python/</tt>) — 7 тестов + 3 графика

Фреймворк: PyCore (TestRunner + DataValidator) Запуск: python test_python/test_discriminators.py
Графики: python test_python/test_discriminators_plot.py
Тесты с визуализацией и сравнением с эталонами NumPy/SciPy.

1.14.4.1 Эталонные реализации

```
# CG — numpy weighted average:
def ref_cg(A, x):
    return np.average(x, weights=A)

# QA — вершина параболы через polyfit:
def ref_qa(A, x):
    coeffs = np.polyfit(x, A, 2)
    return -coeffs[1] / (2.0 * coeffs[0])
```

1.14.4.2 Результат сравнения точности

Метод	Средняя ошибка	Точность
CG	0.172	±0.3 шага
QA	0.007	±0.1 шага

QA в 25 раз точнее CG!

1.14.4.3 Графики

График показывает:

- Голубая линия — sinc(x - 0.25) (ДН антенны, пик смещён на 0.25)
- Жёлтые точки — 3 отсчёта на сетке {-1, 0, +1}
- Вертикальные линии — оценки дискриминаторов
- Зелёная пунктирная — истинное положение пика

График показывает зависимость абсолютной ошибки от смещения пика:

- CG (красный) и SD (фиолетовый) — линейный рост ошибки
- QA (бирюзовый) — ошибка < 0.02 при любом смещении
- EA (жёлтый) — самая маленькая ошибка

Влияние шага сетки на точность:

- При мелкой сетке (< 0.5) все методы точны
- При крупной сетке (> 1.5) EA значительно лучше остальных

1.14.5 Сборка и запуск

1.14.5.1 Простые C++ тесты

```
g++ -std=c++17 -Iinclude test_cpp/test_main.cpp -Lbuild -ldiscr -lm -o test_discr
./test_discr
```

1.14.5.2 Google Test

```
cd build
cmake .. -G Ninja
cmake --build .
./test_discr_gtest
ctest  # или так
```

1.14.5.3 Python

```
python test_python/test_discriminators.py        # sinc(x) тесты
python test_python/test_discriminators_plot.py   # sinc(x) графики → Doc/plots/
python test_python/test_fft_frequency.py         # FFT тесты (10 проверок)
python test_python/test_fft_frequency_plot.py    # FFT графики → Doc/plots/
```

1.14.6 4. FFT-дискриминаторы частоты (<tt>test_python/</tt>) — 10 тестов + 2 графика

Фреймворк: PyCore (TestRunner) Запуск: python test_python/test_fft_frequency.py
Тестирование дискриминаторов в задаче оценки частоты по FFT-спектру (воспроизведение MatLab Primer.m + fcalcdelay.m).

1.14.6.1 Тестовые группы

Класс	Тестов	Что проверяет
TestFFTBasic	4	Симметрия (fsin=0), смещённый пик, все 5 методов
TestFFTSweep	4	Монотонность ДХ, макс. ошибка < 10 кГц, знак оценки
TestFFTCompare	2	EXP лучше SQR лучше LAY; таблица точности

1.14.6.2 Результат

```
[PASS]  TestFFTBasic.test_exp_at_center
[PASS]  TestFFTBasic.test_exp_shifted: f_est=98575, err=4825 Hz
[PASS]  TestFFTSweep.test_exp_monotonic (17 pts)
[PASS]  TestFFTCompare.test_exp_best: EXP(2902) < SQR(13553) < LAY(22933)
Total: 10 passed, 0 failed, 0 skipped
```

1.14.6.3 Графики FFT

1.15 FFT-дискриминаторы частоты

1.15.1 Оценка частоты по FFT-спектру (из Primer.m)

Дискриминаторы модуля применяются не только для оценки угловых координат по ДН антенны, но и для точной оценки частоты по FFT-спектру комплексного сигнала.

> Рекомендация для современного pipeline: для CPU FFT-задачи наилучшим выбором является **discr5ea (МНК-Гауссиан по 5 точкам)** с fallback на discr5qa и обязательным zero-padding ×2 + окном Hanning. Это даёт ошибку ~0.6 Гц при SNR = 20 дБ — в 20 раз лучше argmax и в 2 раза лучше классической 3-точечной EA3. Полное обоснование, дерево решений и Quick reference — на странице Итоговые рекомендации.

1.15.1.1 Сигнал

$$x(t) = A \cdot \exp(j \cdot 2\pi \cdot f_{sin} \cdot t) + \text{noise}$$

Параметр	Значение	Описание
N	32	Число отсчётов
fd	12 МГц	Частота дискретизации
df = fd/N	375 кГц	Ширина бина FFT
fsin	[-df/2, +df/2]	Sweep (±187.5 кГц)
Окно	Hamming	-43 дБ боковые лепестки

1.15.2 5 методов оценки частоты

1.15.2.1 3-точечные (бины k-1, k, k+1)

EXP (discr3ea.c — парабола на log|S|):

$$z_i = \ln|S_i|, \quad \alpha = z_1(f_2^2 - f_3^2) + z_2(f_3^2 - f_1^2) + z_3(f_1^2 - f_2^2)$$

$$\beta = z_1(f_2 - f_3) + z_2(f_3 - f_1) + z_3(f_1 - f_2), \quad f_{est} = \frac{\alpha}{2\beta}$$

SQR (discr3qa.c — парабола на |S|):

$$A_o = \frac{A_2 - A_1}{A_2 - A_3}, \quad f_{est} = \frac{(A_o - 1)f_2^2 - A_o f_3^2 + f_1^2}{2\big((A_o - 1)f_2 - A_o f_3 + f_1\big)}$$

LAY (Jacobsen, fcalcdelay.m):

$$\sigma = \frac{S_{k+1} - S_{k-1}}{2S_k - S_{k-1} - S_{k+1}}, \quad f_{est} = f_k - \mathrm{Re}\{\sigma\} \cdot \frac{f_d}{N}$$

1.15.2.2 2-точечные (top-2 bins по амплитуде)

SD (discrsd.c):

$$f_{est} = \frac{f_1 + f_2}{2} + c \cdot \frac{P_2 - P_1}{P_2 + P_1}, \quad c = 0.132497$$

CG (discrcg.c):

$$f_{est} = \frac{A_1 f_1 + A_2 f_2}{A_1 + A_2}$$

1.15.3 Дискриминационная характеристика

Верхний ряд: спектры FFT при fsin = -df/2, 0, +df/2. Средний ряд: оценки частоты vs. истинная частота (65 точек sweep). Нижний ряд: ошибка оценки.

1.15.4 Сравнение точности

Sweep ±df/4, N=32, окно Hamming, без шума:

Метод	Средняя ошибка	Макс. ошибка	Рекомендация
EXP	2 902 Гц	4 825 Гц	Лучший для Hamming
SQR	13 553 Гц	23 348 Гц	Хороший
LAY	22 933 Гц	43 811 Гц	Biased (Jacobsen для Hamming)
CG	84 664 Гц	115 270 Гц	Грубая оценка
SD	138 393 Гц	187 500 Гц	Очень грубая (c=0.132497)

1.15.5 Влияние оконной функции

Левый график: сравнение всех 5 методов (Hamming). Правый график: метод EXP с окнами Hamming / Hann / Blackman.
Blackman даёт наименьшую ошибку для метода EXP — его форма спектрального пика ближе всего к гауссиане.

1.15.6 Запуск тестов

```
# Тесты (10 проверок):
python test_python/test_fft_frequency.py

# Графики -> Doc/plots/fft_*.png:
python test_python/test_fft_frequency_plot.py
```

1.15.7 Происхождение

```
C-исходники (2010-2012)        MatLab              Python
discr3ea.c (Добродумов) —▶ discr='exp' —▶ fft_discr_exp()
discr3qa.c (Федоров)    —▶ discr='sqr' —▶ fft_discr_sqr()
discrsd.c  (Федоров)    —▶ discr='sd'  —▶ fft_discr_sd()
discrcg.c  (Федоров)    —▶ discr='cg'  —▶ fft_discr_cg()
fcalcdelay.m            —▶ discr='lay' —▶ fft_discr_lay()
```

Глава 2

Алфавитный указатель структур данных

2.1 Структуры данных

Структуры данных с их кратким описанием.

ax_pair_t
Структура для сортировки пар (амплитуда, координата) по x 25
axstr_t
Структура для сортировки пар (амплитуда, координата) 25

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

discr5ea.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. МНК-гауссова аппроксимация по 5 точкам . . . 27

discr_auto.h
Автоматический выбор дискриминатора с экстраполяцией . . . 30

discr_common.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Обертки для дискриминаторных оценок . . . 35

discr_types.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Типы дискриминаторов . . . 43

discrcg.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Центр тяжести . . . 44

discrea.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация экспоненциальной зависимостью . . . 48

discrqa.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация квадратичной зависимостью . . . 52

discrsd.h
Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Суммарно-разностный . . . 54

discr5ea.c
КППО / ТЗК. Дискриминатор — МНК-гауссова аппроксимация по 5 точкам . . . 56

discr_auto.c
Автоматический дискриминатор с экстраполяцией . . . 61

discr_common.c
КППО / ТЗК. Обёртки для дискриминаторных оценок . . . 69

discrcg.c
КППО / ТЗК. Дискриминатор — центр тяжести (Center of Gravity) . . . 77

discrea.c
КППО / ТЗК. Дискриминатор — экспоненциальная аппроксимация (Exponential) 79

discrqa.c
КППО / ТЗК. Дискриминатор — квадратичная аппроксимация (Quadratic) . . . 84

discrsd.c
КППО / ТЗК. Дискриминатор — суммарно-разностный (Sum-Difference) . . . 86

Глава 4

Структуры данных

4.1 Структура ax_pair_t

Структура для сортировки пар (амплитуда, координата) по x.

Поля данных

- double a
- double x

4.1.1 Подробное описание

Структура для сортировки пар (амплитуда, координата) по x.
См. определение в файле discr_auto.c строка 58

4.1.2 Поля

4.1.2.1 a

double ax_pair_t::a
См. определение в файле discr_auto.c строка 59

4.1.2.2 x

double ax_pair_t::x
См. определение в файле discr_auto.c строка 60
Объявления и описания членов структуры находятся в файле:

- discr_auto.c

4.2 Структура axstr_t

Структура для сортировки пар (амплитуда, координата).

Поля данных

- double a

  амплитуда
- double x

  координата

4.2.1 Подробное описание

Структура для сортировки пар (амплитуда, координата).
См. определение в файле discrea.c строка 22

4.2.2 Поля

4.2.2.1 a

double axstr_t::a
амплитуда
См. определение в файле discrea.c строка 23

4.2.2.2 x

double axstr_t::x
координата
См. определение в файле discrea.c строка 24
Объявления и описания членов структуры находятся в файле:

- discrea.c

Глава 5

Файлы

5.1 Файл fft.md

5.2 Файл formulas.md

5.3 Файл mainpage.md

5.4 Файл methods_visual.md

5.5 Файл recommendations.md

5.6 Файл tests.md

5.7 Файл discr5ea.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. МНК-гауссова аппроксимация по 5 точкам.

Функции

- int discr5ea (const double A[5], const double x[5], double *xe)

  Пятиточечный дискриминатор, МНК-гауссова аппроксимация (LSQ Gaussian 5pt).
- int discr5qa (const double A[5], const double x[5], double *xe)

  Пятиточечный дискриминатор, МНК-параболическая аппроксимация (LSQ Parabolic 5pt).

5.7.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. МНК-гауссова аппроксимация по 5 точкам.

Автор

Кодо (AI), Alex

Дата

03.04.2026

См. определение в файле discr5ea.h

5.7.2 Функции

5.7.2.1 discr5ea()

```
int discr5ea (
            const double A[5],
            const double x[5],
            double * xe )
```

Пятиточечный дискриминатор, МНК-гауссова аппроксимация (LSQ Gaussian 5pt).
Аппроксимирует 5 равноотстоящих отсчётов гауссианом методом наименьших квадратов:

$$y = A_{max} \exp\left(-a\,(x - x_0)^2\right)$$

Алгоритм:

1. Логарифмирование: $z_i = \ln A_i$
2. МНК-параболическая аппроксимация $z(x) = a\,x^2 + b\,x + c$ по 5 равноотстоящим точкам (переопределённая система 5 уравнений, 3 неизвестных)
3. Вершина параболы: $x_e = -b/(2a)$

Формулы МНК для нормированных координат $\{-2, -1, 0, 1, 2\}$:

$$a = \frac{2z_1 - z_2 - 2z_3 - z_4 + 2z_5}{14}$$

$$b = \frac{-2z_1 - z_2 + z_4 + 2z_5}{10}$$

$$x_e = x_3 + h \cdot \frac{-b}{2a}$$

Преимущества перед discr3ea:

- Устойчивость к шуму за счёт сглаживания (5 точек → 3 параметра)
- Гауссова модель хорошо приближает sinc(x) вблизи пика

Предупреждения

Все амплитуды должны быть строго положительны ($A_i > 0$).

Точки должны быть равноотстоящими (постоянный шаг h).

Аргументы

A	массив из 5 амплитуд [A1, A2, A3, A4, A5], центральная = A[2]
x	массив из 5 координат [x1, x2, x3, x4, x5], равноотстоящие
xe	[out] оценка координаты x_e

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке (при ошибке *xe содержит приближённое значение)

См. определение в файле discr5ea.c строка 61

```
00062 {
00063   double z[5];
00064   double h, peak;
00065   int i, rc;
00066
00067   /* Шаг сетки (берём из первых двух точек) */
00068   h = x[1] - x[0];
00069   if (fabs(h) < DBL_EPSILON) {
00070     *xe = x[2];
```

```
    return EXIT_FAILURE;
  }

  /* Проверка: все амплитуды положительны */
  for (i = 0; i < 5; i++) {
    if (A[i] < DBL_EPSILON) {
      *xe = x[2];
      return EXIT_FAILURE;
    }
  }

  /* Проверка: не все равны */
  {
    int all_equal = 1;
    for (i = 1; i < 5; i++) {
      if (fabs(A[i] - A[0]) > DBL_EPSILON) {
        all_equal = 0;
        break;
      }
    }
    if (all_equal) {
      *xe = x[2];
      return EXIT_FAILURE;
    }
  }

  /* Логарифмирование */
  for (i = 0; i < 5; i++) {
    z[i] = log(A[i]);
  }

  /* МНК-оценка вершины параболы в лог-масштабе */
  rc = lsq5_peak(z, &peak);

  /* Обратная нормировка: центр = x[2], шаг = h */
  *xe = x[2] + h * peak;

  return rc;
}
```

5.7.2.2 discr5qa()

```
int discr5qa (
            const double A[5],
            const double x[5],
            double * xe )
```

Пятиточечный дискриминатор, МНК-параболическая аппроксимация (LSQ Parabolic 5pt). Аналогичен discr5ea, но работает напрямую с амплитудами (без логарифмирования). Формулы МНК для нормированных координат $\{-2, -1, 0, 1, 2\}$:

$$a = \frac{2A_1 - A_2 - 2A_3 - A_4 + 2A_5}{14}$$

$$b = \frac{-2A_1 - A_2 + A_4 + 2A_5}{10}$$

Аргументы

A	массив из 5 амплитуд [A1, A2, A3, A4, A5]
x	массив из 5 координат [x1, x2, x3, x4, x5], равноотстоящие
xe	[out] оценка координаты x_e

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке

См. определение в файле discr5ea.c строка 112

```
{
  double h, peak;
  int rc;

  /* Шаг сетки */
  h = x[1] - x[0];
```

```
00119   if (fabs(h) < DBL_EPSILON) {
00120     *xe = x[2];
00121     return EXIT_FAILURE;
00122   }
00123
00124   /* Проверка: не все равны */
00125   {
00126     int all_equal = 1;
00127     int i;
00128     for (i = 1; i < 5; i++) {
00129       if (fabs(A[i] - A[0]) > DBL_EPSILON) {
00130         all_equal = 0;
00131         break;
00132       }
00133     }
00134     if (all_equal) {
00135       *xe = x[2];
00136       return EXIT_FAILURE;
00137     }
00138   }
00139
00140   /* МНК-оценка вершины параболы в линейном масштабе */
00141   rc = lsq5_peak(A, &peak);
00142
00143   /* Обратная нормировка */
00144   *xe = x[2] + h * peak;
00145
00146   return rc;
00147 }
```

5.8 discr5ea.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                  (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *****************************************************************************/
00023 #ifndef DISCR_5EA_H
00024 #define DISCR_5EA_H
00025
00026 /*****************************************************************************
00027                        ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00028 *****************************************************************************/
00029
00030 #ifdef __cplusplus
00031 extern "C" {
00032 #endif
00033
00072 int discr5ea(const double A[5], const double x[5], double *xe);
00073
00092 int discr5qa(const double A[5], const double x[5], double *xe);
00093
00094 #ifdef __cplusplus
00095 }
00096 #endif
00097
00098 #endif
```

5.9 Файл discr_auto.h

Автоматический выбор дискриминатора с экстраполяцией.

Функции

- int discr_is_monotonic (double A1, double A2, double A3)

  Проверка монотонности трёх отсчётов.
- int discr3_extrap_gauss (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

Гауссова экстраполяция (E2) для монотонных случаев.

- int discr3_auto (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

  Автоматический дискриминатор с переключением.

5.9.1 Подробное описание

Автоматический выбор дискриминатора с экстраполяцией.

Автор

Кодо (AI Assistant)

Дата

2026-04-02

Решает проблему монотонных данных (пик за сеткой), когда стандартные дискриминаторы (EA, QA, CG) работают плохо (ошибка > 25% шага).
Стратегия:

1. Проверить монотонность 3 отсчётов
2. Если монотонные → Гауссова экстраполяция (E2)
3. Если нормальные → EA (экспоненциальный дискриминатор)
4. Если EA вернул FAILURE → QA (квадратичный fallback)

Обоснование (из аналитического исследования):

- EA: MAE = 0.005 в нормальной зоне (лучший метод)
- E2: MAE = 0.148 для монотонных случаев (лучше E1 и E3)
- QA: MAE = 0.006, fallback если EA не сходится
- Без авто: 67% случаев попадают в монотонную зону!

См. определение в файле discr_auto.h

5.9.2 Функции

5.9.2.1 discr3_auto()

```
int discr3_auto (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3,
            double * xe )
```

Автоматический дискриминатор с переключением.
Алгоритм выбора:

```
if (is_monotonic(A1, A2, A3)):
    xe = extrap_gauss(...)        // E2 — Гауссова экстраполяция
else:
    ret = discr3ea(...)           // EA — экспоненциальный (лучший)
    if (ret == FAILURE):
        xe = discr3qa(...)        // QA — квадратичный (fallback)
```

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки

Аргументы

A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты

Возвращает

0 — EA успех, 1 — использован QA fallback, 2 — использована экстраполяция E2, 3 — экстраполяция E2 не сошлась (xe = x2)

См. определение в файле discr_auto.c строка 191

```
00194 {
00195  int ret;
00196
00197  /*
00198   * Ветка 1: МОНОТОННЫЕ данные (пик за сеткой)
00199   *
00200   * Когда A1 ≥ A2 ≥ A3 или A1 ≤ A2 ≤ A3, среди 3 отсчётов
00201   * нет перегиба — пик ДН находится за пределами сетки.
00202   *
00203   * Стандартные дискриминаторы в этом случае:
00204   *   CG: MAE = 0.46 (почти случайная оценка)
00205   *   QA: MAE = 0.32, MaxErr = 1.23 (парабола улетает!)
00206   *   EA: возвращает FAILURE, ставит xe = краевой отсчёт
00207   *
00208   * Гауссова экстраполяция (E2): MAE = 0.148
00209   *   → в 2-3 раза лучше любого стандартного метода!
00210   */
00211  if (discr_is_monotonic(A1, A2, A3)) {
00212    ret = discr3_extrap_gauss(A1, A2, A3, x1, x2, x3, xe);
00213    return (ret == EXIT_SUCCESS) ? 2 : 3;
00214    /* 2 = E2 сработала, 3 = E2 не смогла (нули, вырожденность) */
00215  }
00216
00217  /*
00218   * Ветка 2: НОРМАЛЬНЫЕ данные (пик внутри сетки)
00219   *
00220   * EA (экспоненциальный): MAE = 0.005 в нормальной зоне
00221   *   Лучший метод! Использует log-параболу, точнее QA.
00222   *
00223   * Если EA вернёт FAILURE (вогнутость, нули), переходим на QA.
00224   * QA: MAE = 0.006 — почти так же точен, но без ограничений
00225   *     на знак амплитуд и выпуклость.
00226   */
00227  ret = discr3ea(A1, A2, A3, x1, x2, x3, xe);
00228
00229  if (ret == EXIT_SUCCESS) {
00230    return 0;  /* EA справился — лучший результат */
00231  }
00232
00233  /*
00234   * Ветка 3: FALLBACK на QA
00235   *
00236   * EA вернул FAILURE. Причины:
00237   *   - Вогнутость (минимум в центре вместо максимума)
00238   *   - Нулевые или отрицательные амплитуды
00239   *   - Числовая вырожденность
00240   *
00241   * QA не требует положительных амплитуд и не проверяет
00242   * выпуклость — работает всегда (кроме совсем вырожденных случаев).
00243   */
00244  *xe = discr3qa(A1, A2, A3, x1, x2, x3);
00245  return 1;  /* QA fallback */
00246 }
```

5.9.2.2 discr3_extrap_gauss()

```
int discr3_extrap_gauss (
                double A1,
                double A2,
                double A3,
                double x1,
```

```
double x2,
double x3,
double * xe )
```

Гауссова экстраполяция (E2) для монотонных случаев.
Алгоритм — как discr3ea, но без проверки выпуклости и с расширенным диапазоном допустимого вылета (2× вместо 0.5× от размера сетки).
Логарифмирует амплитуды, аппроксимирует параболой в log-пространстве, находит вершину. Для монотонных данных вершина за пределами сетки — это ожидаемое и полезное поведение.
Из исследования: E2 MAE = 0.148 для монотонных случаев, что в 2.2 раза лучше E1 (парабола, MAE = 0.324) и в 3.4 раза лучше E3 (градиент, MAE = 0.500).

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
A3	амплитуда третьей точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты (экстраполированная)

Возвращает

0 — успех, 1 — ошибка (нулевые амплитуды, все равны)

См. определение в файле discr_auto.c строка 72

```
00075 {
00076  ax_pair_t pts[3];
00077  double a1, a2, a3, f1, f2, f3;
00078  double z1, z2, z3;
00079  double a_coef, b_coef;
00080  double span;
00081
00082  /*
00083   * Шаг 1. Проверка: все амплитуды > 0
00084   * (нужно для логарифмирования)
00085   */
00086  if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
00087    *xe = x2;
00088    return EXIT_FAILURE;
00089  }
00090
00091  /* Все амплитуды одинаковы → невозможно определить направление */
00092  if (fabs(A1 - A2) < DBL_EPSILON && fabs(A2 - A3) < DBL_EPSILON) {
00093    *xe = x2;
00094    return EXIT_FAILURE;
00095  }
00096
00097  /*
00098   * Шаг 2. Сортировка по координатам (x возрастает)
00099   * Это нужно чтобы формулы работали корректно
00100   * независимо от порядка входных аргументов.
00101   */
00102  pts[0].a = A1; pts[0].x = x1;
00103  pts[1].a = A2; pts[1].x = x2;
00104  pts[2].a = A3; pts[2].x = x3;
00105  qsort(pts, 3, sizeof(ax_pair_t), cmp_by_x);
00106
00107  a1 = pts[0].a; f1 = pts[0].x;
00108  a2 = pts[1].a; f2 = pts[1].x;
00109  a3 = pts[2].a; f3 = pts[2].x;
00110
00111  /*
00112   * Шаг 3. НЕ проверяем выпуклость!
00113   *
00114   * Это ГЛАВНОЕ отличие от discr3ea:
00115   * - discr3ea проверяет что a2 > a1 и a2 > a3 (перегиб)
00116   *   и возвращает FAILURE для монотонных данных
00117   * - Мы РАЗРЕШАЕМ монотонные данные, потому что
00118   *   вершина параболы в log-пространстве покажет
00119   *   КУДА и НАСКОЛЬКО далеко экстраполировать пик.
00120   */
```

```
/*
 * Шаг 4. Логарифмирование
 *
 * Гауссиан y = A*exp(-a*(x-x0)^2) после логарифмирования
 * становится параболой: ln(y) = ln(A) - a*(x-x0)^2
 *
 * Парабола в log-пространстве → вершина = x0 (положение пика).
 */
z1 = log(a1);
z2 = log(a2);
z3 = log(a3);

/*
 * Шаг 5. Коэффициенты параболы
 *
 * Парабола z(x) через 3 точки (f1,z1), (f2,z2), (f3,z3):
 *   a_coef = z1*(f2²-f3²) + z2*(f3²-f1²) + z3*(f1²-f2²)
 *   b_coef = z1*(f2-f3)   + z2*(f3-f1)   + z3*(f1-f2)
 *
 * Вершина: xe = 0.5 * a_coef / b_coef
 */
a_coef = z1 * (f2 * f2 - f3 * f3)
       + z2 * (f3 * f3 - f1 * f1)
       + z3 * (f1 * f1 - f2 * f2);

b_coef = z1 * (f2 - f3)
       + z2 * (f3 - f1)
       + z3 * (f1 - f2);

/* b_coef ≈ 0 → парабола вырожденная, направление неопределено */
if (fabs(b_coef) < DBL_EPSILON) {
  /* Экстраполируем в сторону наибольшей амплитуды */
  if (a3 > a1)
    *xe = f3 + 0.5 * (f3 - f1);
  else
    *xe = f1 - 0.5 * (f3 - f1);
  return EXIT_FAILURE;
}

/* Вершина параболы в log-пространстве = оценка положения пика */
*xe = 0.5 * a_coef / b_coef;

/*
 * Шаг 6. Ограничение вылета (clipping)
 *
 * Расширенный диапазон: ±2 × размер_сетки (вместо ±0.5× в discr3ea).
 * Это позволяет экстраполировать дальше, но не бесконечно далеко.
 *
 * Почему 2×? Из исследования: при вылете > 2× шага ошибка
 * экстраполяции уже не уменьшается, а растёт.
 */
span = f3 - f1;  /* = 2 × шаг_сетки для симметричной сетки */

if (*xe > f3 + span) {
  *xe = f3 + span;
  return EXIT_FAILURE;
}
if (*xe < f1 - span) {
  *xe = f1 - span;
  return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}
```

5.9.2.3 discr_is_monotonic()

```
int discr_is_monotonic (
            double A1,
            double A2,
            double A3 )
```

Проверка монотонности трёх отсчётов.

Монотонные данные = нет локального максимума среди 3 точек. Это означает, что пик за пределами сетки.

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки

Аргументы

A3	амплитуда третьей точки

Возвращает

1 — монотонные (пик за сеткой), 0 — есть перегиб (пик внутри)

См. определение в файле discr_auto.c строка 35

```
00036 {
00037  /*
00038   * Монотонные = все убывают ИЛИ все возрастают.
00039   * Это значит, что среди A1, A2, A3 нет локального максимума,
00040   * т.е. пик ДН находится за пределами сетки из 3 отсчётов.
00041   *
00042   * Примеры:
00043   *   A1=0.8, A2=0.6, A3=0.3 → убывают → монотонные (пик слева)
00044   *   A1=0.3, A2=0.6, A3=0.8 → возрастают → монотонные (пик справа)
00045   *   A1=0.6, A2=0.9, A3=0.7 → перегиб → НЕ монотонные (пик внутри)
00046   */
00047  return (A1 >= A2 && A2 >= A3) ||  /* убывают (или равны) */
00048         (A1 <= A2 && A2 <= A3);     /* возрастают (или равны) */
00049 }
```

5.10 discr_auto.h

См. документацию.

```
00001
00023 #ifndef DISCR_AUTO_H
00024 #define DISCR_AUTO_H
00025
00026 #ifdef __cplusplus
00027 extern "C" {
00028 #endif
00029
00041 int discr_is_monotonic(double A1, double A2, double A3);
00042
00065 int discr3_extrap_gauss(double A1, double A2, double A3,
00066                         double x1, double x2, double x3,
00067                         double *xe);
00068
00093 int discr3_auto(double A1, double A2, double A3,
00094                double x1, double x2, double x3,
00095                double *xe);
00096
00097 #ifdef __cplusplus
00098 }
00099 #endif
00100
00101 #endif /* DISCR_AUTO_H */
```

5.11 Файл discr_common.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Обертки для дискриминаторных оценок.
#include "discr_types.h"

Функции

- int discr2 (double maz[2][2], double mel[2][2], double mval[2][2], double *az, double *el, double *val)

  Двухточечный дискриминатор (обёртка), центр тяжести.
- int discr3_ (double maz[3][3], double mel[3][3], double mval[3][3], DISCRTYPE discrtype, double dx, double lambda, double *az, double *el, double *val)

  Адаптер трехточечного дискриминатора для статических массивов 3x3.
- int discr3 (double **maz, double **mel, double **mval, int ndnx, int ndny, DISCRTYPE discrtype, double dx, double lambda, double *az, double *el, double *val)

  Трехточечный дискриминатор (обёртка).
- int checkuse3d (double **maz, double **mel, double **mval, int ndnx, int ndny)

Проверка на возможность использования 3-точечного дискриминатора.

- int conv3to2 (double maz3[3][3], double mel3[3][3], double mval3[3][3], double maz2[2][2], double mel2[2][2], double mval2[2][2])

  Выделение подматрицы 2x2 из 3x3 для перехода на 2-точечный дискриминатор.

5.11.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Обертки для дискриминаторных оценок.

Автор

Керский Е.В.

Дата

06.07.2017

См. определение в файле discr_common.h

5.11.2 Функции

5.11.2.1 checkuse3d()

```
int checkuse3d (
            double ** maz,
            double ** mel,
            double ** mval,
            int ndnx,
            int ndny )
```

Проверка на возможность использования 3-точечного дискриминатора.
Проверяет, что все амплитуды в матрице $N_x \times N_y$ строго положительны ($A_{ij} > \varepsilon$).

Аргументы

maz	[in] массив азимутальных координат (не используется, для совместимости)
mel	[in] массив угломестных координат (не используется, для совместимости)
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y

Возвращает

1 — можно использовать, 0 — нельзя (есть нулевые/отрицательные амплитуды)

Проверяет что все амплитуды положительны.

Аргументы

maz	[in] массив азимутальных координат (не используется, для совместимости)
mel	[in] массив угломестных координат (не используется, для совместимости)
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y

Возвращает

1 — можно использовать, 0 — нельзя

См. определение в файле discr_common.c строка 192

```
00193 {
00194   int i, j;
00195   (void)maz;
00196   (void)mel;
00197
00198   for (i = 0; i < ndnx; i++)
00199     for (j = 0; j < ndny; j++)
00200       if (mval[i][j] < DBL_EPSILON)  /* ERR-004 fix: was FLT_EPSILON */
00201         return 0;
00202   return 1;
00203 }
```

5.11.2.2 conv3to2()

```
int conv3to2 (
            double maz3[3][3],
            double mel3[3][3],
            double mval3[3][3],
            double maz2[2][2],
            double mel2[2][2],
            double mval2[2][2] )
```

Выделение подматрицы 2x2 из 3x3 для перехода на 2-точечный дискриминатор.
Перебирает все 4 квадранта 2x2 внутри матрицы 3x3 и выбирает тот, в котором суммарная амплитуда максимальна. Используется когда 3-точечный дискриминатор невозможен (checkuse3d() вернул 0).

Аргументы

maz3	[in] азимутальные координаты 3x3
mel3	[in] угломестные координаты 3x3
mval3	[in] амплитуды 3x3
maz2	[out] азимутальные координаты 2x2
mel2	[out] угломестные координаты 2x2
mval2	[out] амплитуды 2x2

Возвращает

0 — успех

Выбирает квадрант 2x2 с максимальной суммарной амплитудой. Используется когда 3-точечный дискриминатор невозможен (checkuse3d вернул 0, вогнутость и т.д.).

Аргументы

maz3	[in] азимутальные координаты 3x3
mel3	[in] угломестные координаты 3x3
mval3	[in] амплитуды 3x3
maz2	[out] азимутальные координаты 2x2
mel2	[out] угломестные координаты 2x2
mval2	[out] амплитуды 2x2

Возвращает

0 — успех

См. определение в файле discr_common.c строка 249

```
{
  int bestR = 0, bestC = 0;
  double bestSum = -1.0;
  int r, c, i, j;

  /* Перебираем все 4 квадранта 2x2 (смещения: (0,0), (0,1), (1,0), (1,1)) */
  for (r = 0; r < 2; r++) {
    for (c = 0; c < 2; c++) {
      double sum = mval3[r][c]     + mval3[r][c + 1]
                 + mval3[r + 1][c] + mval3[r + 1][c + 1];
      if (sum > bestSum) {
        bestSum = sum;
        bestR = r;
        bestC = c;
      }
    }
  }

  /* Копируем выбранный квадрант */
  for (i = 0; i < 2; i++) {
    for (j = 0; j < 2; j++) {
      maz2[i][j]  = maz3[bestR + i][bestC + j];
      mel2[i][j]  = mel3[bestR + i][bestC + j];
      mval2[i][j] = mval3[bestR + i][bestC + j];
    }
  }

  return 0;
}
```

5.11.2.3 discr2()

```
int discr2 (
            double maz[2][2],
            double mel[2][2],
            double mval[2][2],
            double * az,
            double * el,
            double * val )
```

Двухточечный дискриминатор (обёртка), центр тяжести.
Принимает матрицы 2x2 координат и амплитуд ДН. Находит строку/столбец с максимумом и оценивает азимут и угол места через discr2cg().

Аргументы

maz	[in] массив азимутальных координат 2x2
mel	[in] массив угломестных координат 2x2
mval	[in] массив амплитуд 2x2
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка (амплитуда <= 0)

Принимает матрицы 2x2 координат и амплитуд ДН. Находит строку/столбец с максимумом и оценивает az, el через discr2cg().

Аргументы

maz	[in] массив азимутальных координат 2x2

Аргументы

mel	[in] массив угломестных координат 2x2
mval	[in] массив амплитуд 2x2
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка

См. определение в файле discr_common.c строка 43

```
00045 {
00046   int maxRow = 0, maxCol = 0;
00047   double maxAmpl = -1.0;
00048   int i, j;
00049
00050   /* Проверка: все амплитуды должны быть положительны */
00051   for (i = 0; i < 2; ++i)
00052     for (j = 0; j < 2; ++j)
00053       if (mval[i][j] < DBL_EPSILON) {
00054         *az = 0.0; *el = 0.0; *val = 0.0;
00055         return -1;
00056       }
00057
00058   /* Поиск максимума */
00059   for (i = 0; i < 2; ++i)
00060     for (j = 0; j < 2; ++j)
00061       if (mval[i][j] > maxAmpl) {
00062         maxRow = i;
00063         maxCol = j;
00064         maxAmpl = mval[i][j];
00065       }
00066
00067   /* Оценка по азимуту (по строке maxRow) */
00068   *az = discr2cg(mval[maxRow][0], mval[maxRow][1],
00069                  maz[maxRow][0],  maz[maxRow][1]);
00070
00071   /* Оценка по углу места (по столбцу maxCol) */
00072   *el = discr2cg(mval[0][maxCol], mval[1][maxCol],
00073                  mel[0][maxCol],  mel[1][maxCol]);
00074
00075   *val = maxAmpl;
00076   return 0;
00077 }
```

5.11.2.4 discr3()

```
int discr3 (
            double ** maz,
            double ** mel,
            double ** mval,
            int ndnx,
            int ndny,
            DISCRTYPE discrtype,
            double dx,
            double lambda,
            double * az,
            double * el,
            double * val )
```

Трехточечный дискриминатор (обёртка).

Принимает матрицы $N_x \times N_y$ координат и амплитуд ДН, находит строку/столбец с максимумом и вычисляет оценку координат. Для типов DT_QA и DT_EA координаты масштабируются коэффициентом:

$$c = \frac{2\pi \cdot dx}{\lambda}$$

Поддерживает типы: DT_CG, DT_QA, DT_EA.

Предупреждения

Функция НЕ модифицирует входные массивы maz, mel.

Аргументы

maz	[in] массив азимутальных координат
mel	[in] массив угломестных координат
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y
discrtype	[in] тип дискриминатора (DT_CG, DT_QA, DT_EA)
dx	[in] шаг по углу
lambda	[in] длина волны
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — нельзя использовать 3-точ., -2 — неизвестный тип

Принимает матрицы ndnx x ndny координат и амплитуд ДН. Поддерживает типы: DT_CG, DT↩_QA, DT_EA.

Предупреждения

Функция НЕ модифицирует входные массивы maz, mel (исправлено в рефакторинге, ERR-003).

Аргументы

maz	[in] массив азимутальных координат
mel	[in] массив угломестных координат
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y
discrtype	[in] тип дискриминатора (DT_CG, DT_QA, DT_EA)
dx	[in] шаг по углу (используется для DT_QA, DT_EA)
lambda	[in] длина волны (используется для DT_QA, DT_EA)
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — нельзя использовать 3-точ., -2 — неизвестный тип

См. определение в файле discr_common.c строка 101

```
00104 {
00105   double aztmp = 0.0, eltmp = 0.0, valaz = 0.0, valel = 0.0;
```

```
00106  int maxRow = 0, maxCol = 0;
00107  double maxAmpl = -1.0;
00108  const double c = 2.0 * M_PI * dx / lambda;
00109  int i, j;
00110
00111  /* Проверка на возможность использования 3-точечного дискриминатора */
00112  if (!checkuse3d(maz, mel, mval, ndnx, ndny))
00113    return -1;
00114
00115  /* Поиск строки и столбца с максимальной амплитудой */
00116  for (i = 0; i < ndnx; ++i)
00117    for (j = 0; j < ndny; ++j)
00118      if (mval[i][j] > maxAmpl) {
00119        maxRow = i;
00120        maxCol = j;
00121        maxAmpl = mval[i][j];
00122      }
00123
00124  switch (discrtype) {
00125    case DT_CG:
00126      /* ERR-006 fix: добавлена поддержка центра тяжести */
00127      if (ndny >= 3) {
00128        aztmp = discr3cg(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00129                         maz[maxRow][0],  maz[maxRow][1],  maz[maxRow][2]);
00130      }
00131      if (ndnx >= 3) {
00132        eltmp = discr3cg(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00133                         mel[0][maxCol],  mel[1][maxCol],  mel[2][maxCol]);
00134      }
00135      *val = maxAmpl;
00136      break;
00137
00138    case DT_QA:
00139      /* ERR-003 fix: координаты умножаем на c локально, не модифицируя входные данные */
00140      aztmp = discr3qa(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00141                       maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c);
00142      eltmp = discr3qa(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00143                       mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c);
00144      *val = maxAmpl;
00145      /* Обратное преобразование из масштабированных координат */
00146      *az = aztmp / c;
00147      *el = eltmp / c;
00148      return 0;
00149
00150    case DT_EA:
00151      /* ERR-003 fix: координаты умножаем на c локально */
00152      discr3ea(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00153               maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c, &aztmp);
00154      discr3ea(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00155               mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c, &eltmp);
00156
00157      valaz = discr3eaY(mval[maxRow][0], mval[maxRow][1],
00158                        maz[maxRow][0] * c, maz[maxRow][1] * c, aztmp);
00159      valel = discr3eaY(mval[0][maxCol], mval[1][maxCol],
00160                        mel[0][maxCol] * c, mel[1][maxCol] * c, eltmp);
00161
00162      *val = (valaz > valel) ? valaz : valel;
00163      *val = (maxAmpl > *val) ? maxAmpl : *val;
00164      /* Обратное преобразование */
00165      *az = aztmp / c;
00166      *el = eltmp / c;
00167      return 0;
00168
00169    default:
00170      *az = 0.0; *el = 0.0; *val = 0.0;
00171      return -2;
00172  }
00173
00174  /* Для DT_CG — координаты не масштабируются */
00175  *az = aztmp;
00176  *el = eltmp;
00177  return 0;
00178 }
```

5.11.2.5 discr3_()

```
int discr3_ (
            double maz[3][3],
            double mel[3][3],
            double mval[3][3],
            DISCRTYPE discrtype,
            double dx,
            double lambda,
```

double * az,
double * el,
double * val)

Адаптер трехточечного дискриминатора для статических массивов 3x3.
Преобразует массивы double[3][3] в double** и вызывает discr3().

Аргументы

maz	[in] массив азимутальных координат 3x3
mel	[in] массив угломестных координат 3x3
mval	[in] массив амплитуд 3x3
discrtype	[in] тип дискриминатора (DT_CG, DT_QA, DT_EA)
dx	[in] шаг по углу
lambda	[in] длина волны
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка, -2 — неизвестный тип

Адаптер трехточечного дискриминатора для статических массивов 3x3.
Преобразует массивы double[3][3] в double** для вызова discr3().

Аргументы

maz	[in] массив азимутальных координат 3x3
mel	[in] массив угломестных координат 3x3
mval	[in] массив амплитуд 3x3
discrtype	[in] тип дискриминатора
dx	[in] шаг по углу
lambda	[in] длина волны
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка, -2 — неизвестный тип

См. определение в файле discr_common.c строка 221

```
00224 {
00225   double *pmaz[3]  = { &maz[0][0],  &maz[1][0],  &maz[2][0] };
00226   double *pmel[3]  = { &mel[0][0],  &mel[1][0],  &mel[2][0] };
00227   double *pmval[3] = { &mval[0][0], &mval[1][0], &mval[2][0] };
00228
00229   return discr3(pmaz, pmel, pmval, 3, 3,
00230                 discrtype, dx, lambda,
00231                 az, el, val);
00232 }
```

5.12 discr_common.h

См. документацию.

```
00001 /*************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                  (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *************************************************************************/
00023 #ifndef DISCR_COMMON_H
00024 #define DISCR_COMMON_H
00025
00026 #include "discr_types.h"
00027
00028 /*************************************************************************
00029 ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00030 *************************************************************************/
00031
00032 #ifdef __cplusplus
00033 extern "C" {
00034 #endif
00035
00051   int discr2( double maz[ 2 ][ 2 ], double mel[ 2 ][ 2 ], double mval[ 2 ][ 2 ],
00052                double *az, double *el, double *val );
00053
00070   int discr3_( double maz[ 3 ][ 3 ], double mel[ 3 ][ 3 ], double mval[ 3 ][ 3 ],
00071              DISCRTYPE discrtype, double dx, double lambda,
00072              double *az, double *el, double *val );
00073
00101   int discr3( double **maz, double **mel, double **mval, int ndnx, int ndny,
00102             DISCRTYPE discrtype, double dx, double lambda,
00103             double *az, double *el, double *val );
00104
00118   int checkuse3d( double **maz, double **mel, double **mval, int ndnx, int ndny );
00119
00136   int conv3to2( double maz3[ 3 ][ 3 ], double mel3[ 3 ][ 3 ], double mval3[ 3 ][ 3 ],
00137                double maz2[ 2 ][ 2 ], double mel2[ 2 ][ 2 ], double mval2[ 2 ][ 2 ] );
00138
00139 #ifdef __cplusplus
00140 }
00141 #endif
00142
00143 #endif
```

5.13 Файл discr_types.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Типы дискриминаторов.

Перечисления

- enum DISCRTYPE {
  DT_NO = 1 , DT_CG = 2 , DT_SD = 3 , DT_QA = 4 ,
  DT_EA = 5 }

  Типы оценки координат.

5.13.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Типы дискриминаторов.

Автор

Керский Е.В.

Дата

06.07.2017

См. определение в файле discr_types.h

5.13.2 Перечисления

5.13.2.1 DISCRTYPE

enum DISCRTYPE
Типы оценки координат.

Элементы перечислений

DT_NO	Без уточнения.
DT_CG	Центр масс.
DT_SD	Суммарно-разностный.
DT_QA	Квадратичная аппроксимация.
DT_EA	Экспоненциальная аппроксимация.

См. определение в файле discr_types.h строка 29

```
00030 {
00031   DT_NO = 1,
00032   DT_CG = 2,
00033   DT_SD = 3,
00034   DT_QA = 4,
00035   DT_EA = 5,
00036 } DISCRTYPE;
```

5.14 discr_types.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                  (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *****************************************************************************/
00023 #ifndef DISCR_TYPES_H
00024 #define DISCR_TYPES_H
00025
00029 typedef enum
00030 {
00031   DT_NO = 1,
00032   DT_CG = 2,
00033   DT_SD = 3,
00034   DT_QA = 4,
00035   DT_EA = 5,
00036 } DISCRTYPE;
00037
00038 /*****************************************************************************
00039 ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00040 *****************************************************************************/
00041
00042 #ifdef __cplusplus
00043 extern "C" {
00044 #endif
00045
00046
00047 #ifdef __cplusplus
00048 }
00049 #endif
00050
00051 #endif
```

5.15 Файл discrcg.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Центр тяжести.

Функции

- double discr2cg (double A1, double A2, double x1, double x2)

  Двухточечный дискриминатор, центр тяжести.
- double discr3cg (double A1, double A2, double A3, double x1, double x2, double x3)

  Трехточечный дискриминатор, центр тяжести.
- double discrncg (const double *A, const double *x, unsigned int N)

  Дискриминатор для произвольного числа точек, центр тяжести.

5.15.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Центр тяжести.

Автор

Федоров С.А.

Дата

07.05.2010

См. определение в файле discrcg.h

5.15.2 Функции

5.15.2.1 discr2cg()

```
double discr2cg (
            double A1,
            double A2,
            double x1,
            double x2 )
```

Двухточечный дискриминатор, центр тяжести.
Формула: $x_e = \frac{A_1 \cdot x_1 + A_2 \cdot x_2}{A_1 + A_2}$

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты; при A1+A2=0 возвращает (x1+x2)/2

Формула: xe = (A1*x1 + A2*x2) / (A1 + A2)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты; при A1+A2=0 возвращает середину (x1+x2)/2

См. определение в файле discrcg.c строка 28

```
00030 {
00031   double sum = A1 + A2;
00032   if (fabs(sum) < DBL_EPSILON)
00033     return (x1 + x2) * 0.5;
00034   return (A1 * x1 + A2 * x2) / sum;
00035 }
```

5.15.2.2 discr3cg()

```
double discr3cg (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3 )
```

Трехточечный дискриминатор, центр тяжести.
Формула: $x_e = \frac{A_1x_1+A_2x_2+A_3x_3}{A_1+A_2+A_3}$

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты; при сумме=0 возвращает x2

Формула: xe = (A1∗x1 + A2∗x2 + A3∗x3) / (A1 + A2 + A3)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты; при сумме=0 возвращает x2 (центральная точка)

См. определение в файле discrcg.c строка 50

```
00052 {
00053   double sum = A1 + A2 + A3;
00054   if (fabs(sum) < DBL_EPSILON)
00055     return x2;
00056   return (A1 * x1 + A2 * x2 + A3 * x3) / sum;
00057 }
```

5.15.2.3 discrncg()

double discrncg (
const double * A,
const double * x,
unsigned int N)

Дискриминатор для произвольного числа точек, центр тяжести.
Формула: $x_e = \frac{\sum_{i=0}^{N-1} A_i \cdot x_i}{\sum_{i=0}^{N-1} A_i}$

Аргументы

A	массив амплитуд (N элементов)
x	массив координат (N элементов)
N	число точек

Возвращает

оценка координаты; при sum(A)=0 возвращает x[N/2]

Формула: xe = sum(A[i]*x[i]) / sum(A[i])

Аргументы

A	массив амплитуд точек (N элементов)
x	массив координат точек (N элементов)
N	число точек

Возвращает

оценка координаты; при sum(A)=0 возвращает x[N/2]

См. определение в файле discrcg.c строка 69

```
00070 {
00071   unsigned int n;
00072   double Axs = 0.0, As = 0.0;
00073
00074   for (n = 0; n < N; n++) {
00075     As += A[n];
00076     Axs += A[n] * x[n];
00077   }
00078
00079   if (fabs(As) < DBL_EPSILON) {
00080     if (N > 0)
00081       return x[N / 2];
00082     return 0.0;
00083   }
00084   return Axs / As;
00085 }
```

5.16 discrcg.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                    (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 ******************************************************************************/
00023 #ifndef DISCR_CG_H
```

```
#define DISCR_CG_H

/*****************************************************************************
                    ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
*****************************************************************************/

#ifdef __cplusplus
extern "C" {
#endif

double discr2cg(double A1, double A2,
                double x1, double x2);

double discr3cg(double A1, double A2, double A3,
                double x1, double x2, double x3);

double discrncg(const double *A, const double *x, unsigned int N);

#ifdef __cplusplus
}
#endif

#endif
```

5.17 Файл discrea.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация экспоненциальной зависимостью.

Функции

- int discr3ea (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

  Трехточечный дискриминатор, экспоненциальная аппроксимация (Exponential).
- double discr3eaY (double A1, double A2, double x1, double x2, double xe)

  Уточнение амплитуды по экспоненциальной аппроксимации.

5.17.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация экспоненциальной зависимостью.

Автор

Добродумов А.Б.

Дата

15.04.2012

См. определение в файле discrea.h

5.17.2 Функции

5.17.2.1 discr3ea()

```
int discr3ea (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3,
            double * xe )
```

Трехточечный дискриминатор, экспоненциальная аппроксимация (Exponential).
Аппроксимирует отсчёты ДН функцией:

$$y = A_{max} \exp\left(-a\,(x - x_0)^2\right)$$

Алгоритм: логарифмирование амплитуд $z_i = \ln A_i$, аппроксимация параболой $z(x) = \alpha + \beta x + \gamma x^2$ и нахождение вершины:

$$x_e = \frac{\alpha}{2\,\beta}$$

где α и β вычисляются из системы уравнений по трём логарифмированным амплитудам.

Предупреждения

Все амплитуды должны быть строго положительны ($A_i > 0$). Максимум должен лежать в центре (выпуклость).

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
A3	амплитуда третьей точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты x_e

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке (при ошибке ∗xe содержит приближённое значение)

Трехточечный дискриминатор, экспоненциальная аппроксимация (Exponential).
Алгоритм:

1. Проверка входных данных (ненулевые амплитуды, не все равны)
2. Сортировка по координате (x должна возрастать)
3. Проверка на выпуклость (максимум должен быть в центре)
4. Логарифмирование: z = ln(A), аппроксимация параболой z(x)
5. Вершина параболы: xe = 0.5 ∗ a / b
6. Ограничение на вылет за пределы зондирования

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
A3	амплитуда третьей точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке (при ошибке ∗xe содержит приближённое значение)

См. определение в файле discrea.c строка 50

```
00052 {
00053   double a = 0.0, b = 0.0;
00054   double z1, z2, z3;
00055   double f1, f2, f3;
00056   double a1, a2, a3, a12, a23, a13;
00057   /* ERR-002 fix: локальная переменная вместо static */
00058   axstr_t ax[3];
00059
00060   /* Проверка: все амплитуды должны быть положительны */
00061   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
00062     *xe = x2;
00063     return EXIT_FAILURE;
00064   }
00065
00066   /* Проверка: не все амплитуды равны */
00067   if ((fabs(A1 - A2) < DBL_EPSILON) && (fabs(A2 - A3) < DBL_EPSILON)) {
00068     *xe = x2;
00069     return EXIT_FAILURE;
00070   }
00071
00072   /* Сортировка по координате (x должна возрастать) */
00073   ax[0].a = A1; ax[0].x = x1;
00074   ax[1].a = A2; ax[1].x = x2;
00075   ax[2].a = A3; ax[2].x = x3;
00076   qsort(ax, 3, sizeof(axstr_t), compare_by_x);
00077
00078   a1 = ax[0].a; a2 = ax[1].a; a3 = ax[2].a;
00079   f1 = ax[0].x; f2 = ax[1].x; f3 = ax[2].x;
00080
00081   /* Проверка на выпуклость: максимум должен быть в центре */
00082   a12 = a1 - a2;
00083   a23 = a2 - a3;
00084   a13 = a1 - a3;
00085
00086   /* Вогнутость: максимум справа (например 2, 4, 8) */
00087   if ((a12 < 0 && a23 < 0 && a12 > a23) ||
00088       (a12 > 0 && a23 < 0 && a13 < 0)) {
00089     *xe = x3;
00090     return EXIT_FAILURE;
00091   }
00092   /* Вогнутость: максимум слева (например 8, 4, 2) */
00093   if ((a12 > 0 && a23 > 0 && a12 > a23) ||
00094       (a12 > 0 && a23 < 0 && a13 > 0)) {
00095     *xe = x1;
00096     return EXIT_FAILURE;
00097   }
00098
00099   /* Логарифмирование */
00100   z1 = log(a1);
00101   z2 = log(a2);
00102   z3 = log(a3);
00103
00104   /* Коэффициенты параболы z(x) = a + b*x + ... */
00105   a = z1 * (f2 * f2 - f3 * f3) + z2 * (f3 * f3 - f1 * f1) + z3 * (f1 * f1 - f2 * f2);
00106   b = z1 * (f2 - f3) + z2 * (f3 - f1) + z3 * (f1 - f2);
00107
00108   if (fabs(b) < DBL_EPSILON) {
00109     if (a * b >= 0) {
00110       *xe = f3 + 0.5 * (f3 - f1);
00111       return EXIT_FAILURE;
00112     } else {
00113       *xe = f1 - 0.5 * (f3 - f1);
00114       return EXIT_FAILURE;
00115     }
00116   }
00117
00118   /* Вершина параболы */
00119   *xe = 0.5 * a / b;
00120
00121   /* Ограничение на вылет за пределы зондирования */
00122   if (*xe > f3 + 0.5 * (f3 - f1)) {
00123     *xe = f3 + 0.5 * (f3 - f1);
00124     return EXIT_FAILURE;
00125   }
00126   if (*xe < f1 - 0.5 * (f3 - f1)) {
00127     *xe = f1 - 0.5 * (f3 - f1);
00128     return EXIT_FAILURE;
00129   }
00130
00131   return EXIT_SUCCESS;
00132 }
```

5.17.2.2 discr3eaY()

double discr3eaY (

double A1,
double A2,
double x1,
double x2,
double xe)

Уточнение амплитуды по экспоненциальной аппроксимации.
По двум точкам и найденной координате x_e вычисляет уточнённую амплитуду в точке x_e:

$$y_e = A_2 \cdot \exp\bigl(a_0 \cdot (x_2 - x_e)^2\bigr)$$

где коэффициент a_0 определяется из соотношения:

$$a_0 = \frac{\ln A_1 - \ln A_2}{2\,x_e\,(x_1 - x_2) - x_1^2 + x_2^2}$$

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
xe	вычисленная координата (результат discr3ea)

Возвращает

уточнённое значение амплитуды y_e; при $A_1 < \varepsilon$ или $A_2 < \varepsilon$ возвращает 0.0

Формула: ye = A2 ∗ exp(a0 ∗ (x2 - xe)^2)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки
xe	вычисленная координата (результат discr3ea)

Возвращает

уточнённое значение амплитуды

См. определение в файле discrea.c строка 146

```
00147 {
00148   double ye, yMax;
00149   double z1, z2;
00150   double a0, dxe;
00151
00152   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON)
00153     return 0.0;
00154
00155   yMax = (A1 > A2) ? A1 : A2;
00156
00157   z1 = log(A1);
00158   z2 = log(A2);
00159
00160   a0 = (2.0 * xe * (x1 - x2) - x1 * x1 + x2 * x2);
00161
00162   if (fabs(a0) < DBL_EPSILON)
00163     return yMax;
00164
00165   a0 = (z1 - z2) / a0;
00166
```

```
00167   dxe = x2 - xe;
00168   ye = A2 * exp(a0 * dxe * dxe);
00169
00170   return ye;
00171 }
```

5.18 discrea.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                    (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *****************************************************************************/
00023 #ifndef DISCR_EA_H
00024 #define DISCR_EA_H
00025
00026 /*****************************************************************************
00027                        ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00028 *****************************************************************************/
00029
00030 #ifdef __cplusplus
00031 extern "C" {
00032 #endif
00033
00064 int discr3ea(double A1, double A2, double A3,
00065              double x1, double x2, double x3, double* xe);
00066
00090 double discr3eaY(double A1, double A2, double x1, double x2, double xe);
00091
00092 #ifdef __cplusplus
00093 }
00094 #endif
00095
00096 #endif
```

5.19 Файл discrqa.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация квадратичной зависимостью.

Функции

- double discr3qa (double A1, double A2, double A3, double x1, double x2, double x3)

  Трехточечный дискриминатор, аппроксимация квадратичной зависимостью (Quadratic).

5.19.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Аппроксимация квадратичной зависимостью.

Автор

Федоров С.А.

Дата

07.05.2010

См. определение в файле discrqa.h

5.19.2 Функции

5.19.2.1 discr3qa()

```
double discr3qa (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3 )
```

Трехточечный дискриминатор, аппроксимация квадратичной зависимостью (Quadratic).
Аппроксимирует 3 точки параболой и находит вершину. Вспомогательный коэффициент:

$$A_o = \frac{A_2 - A_1}{A_2 - A_3}$$

Формула оценки координаты:

$$x_e = \frac{1}{2} \cdot \frac{(A_o - 1)\, x_2^2 - A_o\, x_3^2 + x_1^2}{(A_o - 1)\, x_2 - A_o\, x_3 + x_1}$$

Граничные случаи:

- $A_2 = A_3$: если $A_1 = A_2$ возвращает x_2; если $A_1 > A_2$ возвращает x_1; иначе $(x_2 + x_3)/2$
- $A_1 = A_2$: если $A_3 > A_2$ возвращает x_3; иначе $(x_1 + x_2)/2$

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты x_e

Трехточечный дискриминатор, аппроксимация квадратичной зависимостью (Quadratic).
Аппроксимирует 3 точки параболой y = a∗x^2 + b∗x + c и находит вершину.
Формулы: Ao = (A2 - A1) / (A2 - A3) xe = 0.5 ∗ ((Ao-1)∗x2^2 - Ao∗x3^2 + x1^2) / ((Ao-1)∗x2 - Ao∗x3 + x1)
Граничные случаи:

- A2 = A3: если A1 = A2 → x2; если A1 > A2 → x1; иначе → (x2+x3)/2
- A1 = A2: если A3 > A2 → x3; иначе → (x1+x2)/2

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты

См. определение в файле discrqa.c строка 39

```
00041 {
00042   if (fabs(A2 - A3) < DBL_EPSILON) {
00043     if (fabs(A2 - A1) < DBL_EPSILON)
00044       return x2;
00045     if (A1 > A2)
00046       return x1;
00047     else
00048       return 0.5 * (x3 + x2);
00049   }
00050
00051   if (fabs(A2 - A1) < DBL_EPSILON) {
00052     if (A3 > A2)
00053       return x3;
00054     else
00055       return 0.5 * (x1 + x2);
00056   }
00057
00058   {
00059     const double Ao = (A2 - A1) / (A2 - A3);
00060     double denom = (Ao - 1.0) * x2 - Ao * x3 + x1;
00061
00062     if (fabs(denom) < DBL_EPSILON)
00063       return x2;
00064
00065     return 0.5 * ((Ao - 1.0) * x2 * x2 - Ao * x3 * x3 + x1 * x1) / denom;
00066   }
00067 }
```

5.20 discrqa.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                  (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *****************************************************************************/
00023 #ifndef DISCR_QA_H
00024 #define DISCR_QA_H
00025
00026 /*****************************************************************************
00027                        ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00028 *****************************************************************************/
00029
00030 #ifdef __cplusplus
00031 extern "C" {
00032 #endif
00033
00065 double discr3qa(double A1, double A2, double A3,
00066                 double x1, double x2, double x3);
00067
00068 #ifdef __cplusplus
00069 }
00070 #endif
00071
00072 #endif
```

5.21 Файл discrsd.h

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Суммарно-разностный.

Функции

- double discr2sd (double c, double A1, double A2, double x1, double x2)

  Двухточечный дискриминатор, суммарно-разностный (Sum-Difference).

5.21.1 Подробное описание

Комплекс программ первичной обработки (КППО). Программа вычисления текущих замеров координат (ТЗК). Дискриминатор. Суммарно-разностный.

Автор

Федоров С.А.

Дата

07.05.2010

См. определение в файле discrsd.h

5.21.2 Функции

5.21.2.1 discr2sd()

```
double discr2sd (
            double c,
            double A1,
            double A2,
            double x1,
            double x2 )
```

Двухточечный дискриминатор, суммарно-разностный (Sum-Difference).
Формула:

$$x_e = \frac{x_1 + x_2}{2} + c \cdot \frac{A_2 - A_1}{A_2 + A_1}$$

Аргументы

c	коэффициент суммарно-разностного дискриминатора
A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты x_e; при $A_1 + A_2 = 0$ возвращает середину $(x_1 + x_2)/2$

Двухточечный дискриминатор, суммарно-разностный (Sum-Difference).
Формулы: xc = (x1 + x2) / 2 dx = c ∗ (A2 - A1) / (A2 + A1) xe = xc + dx

Аргументы

c	коэффициент суммарно-разностного дискриминатора
A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты; при A1+A2=0 возвращает середину (x1+x2)/2

См. определение в файле discrsd.c строка 32

```
00034 {
00035   double xc = (x1 + x2) * 0.5;
00036   double sum = A2 + A1;
00037
00038   if (fabs(sum) < DBL_EPSILON)
00039     return xc;
00040
00041   double dx = c * ((A2 - A1) / sum);
00042   return xc + dx;
00043 }
```

5.22 discrsd.h

См. документацию.

```
00001 /*****************************************************************************
00002 *
00003 *                          ОАО НПК НИИДАР
00004 *                    107258, Москва, Бухвостова 11/12
00005 *
00006 *                  (c)Copyright 2009, НИИДАР, Москва
00007 *
00008 * Все права защищены.  Данная программа не предназначена для публикации  и
00009 * копирования.  Программа состоит из конфиденциальных, содержащих
00010 * коммерческую тайну НИИДАРа материалов. Любые попытки или участие
00011 * в дешифрование, перепрограммировании и  ином изменении кода программы
00012 * строго запрещены без письменного разрешения руководства НИИДАРа.
00013 *****************************************************************************/
00023 #ifndef DISCR_SD_H
00024 #define DISCR_SD_H
00025
00026 /*****************************************************************************
00027                      ОБЪЯВЛЕНИЕ ПРОТОТИПОВ ФУНКЦИЙ
00028 *****************************************************************************/
00029
00030 #ifdef __cplusplus
00031 extern "C" {
00032 #endif
00033
00050 double discr2sd(double c, double A1, double A2,
00051                double x1, double x2);
00052
00053 #ifdef __cplusplus
00054 }
00055 #endif
00056
00057
00058 #endif
```

5.23 Файл discr5ea.c

КППО / ТЗК. Дискриминатор — МНК-гауссова аппроксимация по 5 точкам.

#include "discr5ea.h"
#include <float.h>
#include <math.h>
#include <stdlib.h>

Функции

- static int lsq5_peak (const double y[5], double *peak)

  Внутренняя функция: МНК-параболическая оценка вершины по 5 значениям yi.
- int discr5ea (const double A[5], const double x[5], double *xe)

  Пятиточечный дискриминатор, МНК-гауссова аппроксимация (LSQ Gaussian 5pt).
- int discr5qa (const double A[5], const double x[5], double *xe)

  Пятиточечный дискриминатор, МНК-параболическая аппроксимация (LSQ Parabolic 5pt).

5.23.1 Подробное описание

КППО / ТЗК. Дискриминатор — МНК-гауссова аппроксимация по 5 точкам.

Автор

Кодо (AI), Alex

Дата

03.04.2026

Реализует два метода МНК по 5 равноотстоящим точкам:

- discr5ea: гауссов (в лог-масштабе)
- discr5qa: параболический (в линейном масштабе)

Математика: Нормируем координаты xi -> {-2, -1, 0, 1, 2}. Для переопределённой системы y = a∗x^2 + b∗x + c (5 уравнений, 3 неизвестных) нормальные уравнения (A^T ∗ A) ∗ beta = A^T ∗ y дают замкнутые формулы:
a = (2∗y1 - y2 - 2∗y3 - y4 + 2∗y5) / 14 b = (-2∗y1 - y2 + y4 + 2∗y5) / 10 xe = -b / (2∗a) (в нормированных единицах)
Затем обратная нормировка: xe_real = x3 + h ∗ xe_norm
См. определение в файле discr5ea.c

5.23.2 Функции

5.23.2.1 discr5ea()

```
int discr5ea (
            const double A[5],
            const double x[5],
            double * xe )
```

Пятиточечный дискриминатор, МНК-гауссова аппроксимация (LSQ Gaussian 5pt).
Аппроксимирует 5 равноотстоящих отсчётов гауссианом методом наименьших квадратов:

$$y = A_{max} \exp\left(-a\,(x - x_0)^2\right)$$

Алгоритм:

1. Логарифмирование: $z_i = \ln A_i$
2. МНК-параболическая аппроксимация $z(x) = a\,x^2 + b\,x + c$ по 5 равноотстоящим точкам (переопределённая система 5 уравнений, 3 неизвестных)
3. Вершина параболы: $x_e = -b/(2a)$

Формулы МНК для нормированных координат $\{-2, -1, 0, 1, 2\}$:

$$a = \frac{2z_1 - z_2 - 2z_3 - z_4 + 2z_5}{14}$$

$$b = \frac{-2z_1 - z_2 + z_4 + 2z_5}{10}$$

$$x_e = x_3 + h \cdot \frac{-b}{2a}$$

Преимущества перед discr3ea:

- Устойчивость к шуму за счёт сглаживания (5 точек → 3 параметра)
- Гауссова модель хорошо приближает sinc(x) вблизи пика

Предупреждения

Все амплитуды должны быть строго положительны ($A_i > 0$).

Точки должны быть равноотстоящими (постоянный шаг h).

Аргументы

A	массив из 5 амплитуд [A1, A2, A3, A4, A5], центральная = A[2]
x	массив из 5 координат [x1, x2, x3, x4, x5], равноотстоящие
xe	[out] оценка координаты x_e

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке (при ошибке *xe содержит приближённое значение)

См. определение в файле discr5ea.c строка 61

```
00062 {
00063   double z[5];
00064   double h, peak;
00065   int i, rc;
00066
00067   /* Шаг сетки (берём из первых двух точек) */
00068   h = x[1] - x[0];
00069   if (fabs(h) < DBL_EPSILON) {
00070     *xe = x[2];
00071     return EXIT_FAILURE;
00072   }
00073
00074   /* Проверка: все амплитуды положительны */
00075   for (i = 0; i < 5; i++) {
00076     if (A[i] < DBL_EPSILON) {
00077       *xe = x[2];
00078       return EXIT_FAILURE;
00079     }
00080   }
00081
00082   /* Проверка: не все равны */
00083   {
00084     int all_equal = 1;
00085     for (i = 1; i < 5; i++) {
00086       if (fabs(A[i] - A[0]) > DBL_EPSILON) {
00087         all_equal = 0;
00088         break;
00089       }
00090     }
00091     if (all_equal) {
00092       *xe = x[2];
00093       return EXIT_FAILURE;
00094     }
00095   }
00096
00097   /* Логарифмирование */
00098   for (i = 0; i < 5; i++) {
00099     z[i] = log(A[i]);
00100   }
00101
00102   /* МНК-оценка вершины параболы в лог-масштабе */
00103   rc = lsq5_peak(z, &peak);
00104
00105   /* Обратная нормировка: центр = x[2], шаг = h */
00106   *xe = x[2] + h * peak;
00107
00108   return rc;
00109 }
```

5.23.2.2 discr5qa()

```
int discr5qa (
            const double A[5],
            const double x[5],
            double * xe )
```

Пятиточечный дискриминатор, МНК-параболическая аппроксимация (LSQ Parabolic 5pt).
Аналогичен discr5ea, но работает напрямую с амплитудами (без логарифмирования).
Формулы МНК для нормированных координат $\{-2, -1, 0, 1, 2\}$:

$$a = \frac{2A_1 - A_2 - 2A_3 - A_4 + 2A_5}{14}$$

$$b = \frac{-2A_1 - A_2 + A_4 + 2A_5}{10}$$

Аргументы

A	массив из 5 амплитуд [A1, A2, A3, A4, A5]
x	массив из 5 координат [x1, x2, x3, x4, x5], равноотстоящие
xe	[out] оценка координаты x_e

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке

См. определение в файле discr5ea.c строка 112

```
00113 {
00114   double h, peak;
00115   int rc;
00116
00117   /* Шаг сетки */
00118   h = x[1] - x[0];
00119   if (fabs(h) < DBL_EPSILON) {
00120     *xe = x[2];
00121     return EXIT_FAILURE;
00122   }
00123
00124   /* Проверка: не все равны */
00125   {
00126     int all_equal = 1;
00127     int i;
00128     for (i = 1; i < 5; i++) {
00129       if (fabs(A[i] - A[0]) > DBL_EPSILON) {
00130         all_equal = 0;
00131         break;
00132       }
00133     }
00134     if (all_equal) {
00135       *xe = x[2];
00136       return EXIT_FAILURE;
00137     }
00138   }
00139
00140   /* МНК-оценка вершины параболы в линейном масштабе */
00141   rc = lsq5_peak(A, &peak);
00142
00143   /* Обратная нормировка */
00144   *xe = x[2] + h * peak;
00145
00146   return rc;
00147 }
```

5.23.2.3 lsq5_peak()

static int lsq5_peak (
const double y[5],
double * peak) [static]

Внутренняя функция: МНК-параболическая оценка вершины по 5 значениям yi.
Координаты предполагаются нормированными {-2, -1, 0, 1, 2}.

Аргументы

y	массив из 5 значений (амплитуды или логарифмы)
peak	[out] смещение вершины в нормированных координатах

Возвращает

EXIT_SUCCESS или EXIT_FAILURE

См. определение в файле discr5ea.c строка 36

```
00037 {
00038   /* МНК-коэффициенты параболы y = a*t^2 + b*t + c
00039    * для t = {-2, -1, 0, 1, 2} */
00040   double a = (2.0 * y[0] - y[1] - 2.0 * y[2] - y[3] + 2.0 * y[4]) / 14.0;
00041   double b = (-2.0 * y[0] - y[1] + y[3] + 2.0 * y[4]) / 10.0;
```

```
00042
00043   /* Парабола вогнутая (a < 0) — есть максимум */
00044   if (a > -DBL_EPSILON) {
00045     *peak = 0.0;
00046     return EXIT_FAILURE;
00047   }
00048
00049   *peak = -b / (2.0 * a);
00050
00051   /* Ограничение: вершина не дальше ±2 шагов от центра */
00052   if (*peak < -2.0)
00053     *peak = -2.0;
00054   else if (*peak > 2.0)
00055     *peak = 2.0;
00056
00057   return EXIT_SUCCESS;
00058 }
```

5.24 discr5ea.c

См. документацию.

```
00001
00023 #include "discr5ea.h"
00024 #include <float.h>
00025 #include <math.h>
00026 #include <stdlib.h>
00027
00036 static int lsq5_peak(const double y[5], double *peak)
00037 {
00038   /* МНК-коэффициенты параболы y = a*t^2 + b*t + c
00039    * для t = {-2, -1, 0, 1, 2} */
00040   double a = (2.0 * y[0] - y[1] - 2.0 * y[2] - y[3] + 2.0 * y[4]) / 14.0;
00041   double b = (-2.0 * y[0] - y[1] + y[3] + 2.0 * y[4]) / 10.0;
00042
00043   /* Парабола вогнутая (a < 0) — есть максимум */
00044   if (a > -DBL_EPSILON) {
00045     *peak = 0.0;
00046     return EXIT_FAILURE;
00047   }
00048
00049   *peak = -b / (2.0 * a);
00050
00051   /* Ограничение: вершина не дальше ±2 шагов от центра */
00052   if (*peak < -2.0)
00053     *peak = -2.0;
00054   else if (*peak > 2.0)
00055     *peak = 2.0;
00056
00057   return EXIT_SUCCESS;
00058 }
00059
00060
00061 int discr5ea(const double A[5], const double x[5], double *xe)
00062 {
00063   double z[5];
00064   double h, peak;
00065   int i, rc;
00066
00067   /* Шаг сетки (берём из первых двух точек) */
00068   h = x[1] - x[0];
00069   if (fabs(h) < DBL_EPSILON) {
00070     *xe = x[2];
00071     return EXIT_FAILURE;
00072   }
00073
00074   /* Проверка: все амплитуды положительны */
00075   for (i = 0; i < 5; i++) {
00076     if (A[i] < DBL_EPSILON) {
00077       *xe = x[2];
00078       return EXIT_FAILURE;
00079     }
00080   }
00081
00082   /* Проверка: не все равны */
00083   {
00084     int all_equal = 1;
00085     for (i = 1; i < 5; i++) {
00086       if (fabs(A[i] - A[0]) > DBL_EPSILON) {
00087         all_equal = 0;
00088         break;
00089       }
00090     }
00091     if (all_equal) {
00092       *xe = x[2];
```

```
      return EXIT_FAILURE;
    }
  }

  /* Логарифмирование */
  for (i = 0; i < 5; i++) {
    z[i] = log(A[i]);
  }

  /* МНК-оценка вершины параболы в лог-масштабе */
  rc = lsq5_peak(z, &peak);

  /* Обратная нормировка: центр = x[2], шаг = h */
  *xe = x[2] + h * peak;

  return rc;
}


int discr5qa(const double A[5], const double x[5], double *xe)
{
  double h, peak;
  int rc;

  /* Шаг сетки */
  h = x[1] - x[0];
  if (fabs(h) < DBL_EPSILON) {
    *xe = x[2];
    return EXIT_FAILURE;
  }

  /* Проверка: не все равны */
  {
    int all_equal = 1;
    int i;
    for (i = 1; i < 5; i++) {
      if (fabs(A[i] - A[0]) > DBL_EPSILON) {
        all_equal = 0;
        break;
      }
    }
    if (all_equal) {
      *xe = x[2];
      return EXIT_FAILURE;
    }
  }

  /* МНК-оценка вершины параболы в линейном масштабе */
  rc = lsq5_peak(A, &peak);

  /* Обратная нормировка */
  *xe = x[2] + h * peak;

  return rc;
}
```

5.25 Файл discr_auto.c

Автоматический дискриминатор с экстраполяцией.

#include "discr_auto.h"
#include "discrea.h"
#include "discrqa.h"
#include <math.h>
#include <float.h>
#include <stdlib.h>

Структуры данных

- struct ax_pair_t

  Структура для сортировки пар (амплитуда, координата) по x.

Функции

- int discr_is_monotonic (double A1, double A2, double A3)

  Проверка монотонности трёх отсчётов.

- static int cmp_by_x (const void *p1, const void *p2)
- int discr3_extrap_gauss (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

  Гауссова экстраполяция (E2) для монотонных случаев.
- int discr3_auto (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

  Автоматический дискриминатор с переключением.

5.25.1 Подробное описание

Автоматический дискриминатор с экстраполяцией.

Автор

Кодо (AI Assistant)

Дата

2026-04-02

Реализует стратегию автопереключения на основе результатов аналитического исследования (analysis/REPORT.md):
Нормальный случай (есть перегиб): EA → QA fallback EA даёт MAE = 0.005 (0.5% шага сетки)
Монотонный случай (пик за сеткой): E2 (Гауссова экстраполяция) E2 даёт MAE = 0.148, что в 2-3 раза лучше прямого EA
E2 — это тот же алгоритм что и EA (логарифмирование + парабола), но БЕЗ проверки на выпуклость и с расширенным clipping (2× вместо 0.5×). Это позволяет вершине параболы оказаться далеко за сеткой — что и нужно.
См. определение в файле discr_auto.c

5.25.2 Функции

5.25.2.1 cmp_by_x()

```
static int cmp_by_x (
            const void * p1,
            const void * p2 )   [static]
```

См. определение в файле discr_auto.c строка 63

```
00064 {
00065   const ax_pair_t *a = (const ax_pair_t *)p1;
00066   const ax_pair_t *b = (const ax_pair_t *)p2;
00067   if (a->x < b->x) return -1;
00068   if (a->x > b->x) return  1;
00069   return 0;
00070 }
```

5.25.2.2 discr3_auto()

```
int discr3_auto (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3,
            double * xe )
```

Автоматический дискриминатор с переключением.
Алгоритм выбора:

```
if (is_monotonic(A1, A2, A3)):
   xe = extrap_gauss(...)        // E2 — Гауссова экстраполяция
else:
   ret = discr3ea(...)           // EA — экспоненциальный (лучший)
   if (ret == FAILURE):
      xe = discr3qa(...)         // QA — квадратичный (fallback)
```

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты

Возвращает

0 — EA успех, 1 — использован QA fallback, 2 — использована экстраполяция E2, 3 — экстраполяция E2 не сошлась (xe = x2)

См. определение в файле discr_auto.c строка 191

```
00194 {
00195  int ret;
00196
00197  /*
00198   * Ветка 1: МОНОТОННЫЕ данные (пик за сеткой)
00199   *
00200   * Когда A1 ≥ A2 ≥ A3 или A1 ≤ A2 ≤ A3, среди 3 отсчётов
00201   * нет перегиба — пик ДН находится за пределами сетки.
00202   *
00203   * Стандартные дискриминаторы в этом случае:
00204   *   CG: MAE = 0.46 (почти случайная оценка)
00205   *   QA: MAE = 0.32, MaxErr = 1.23 (парабола улетает!)
00206   *   EA: возвращает FAILURE, ставит xe = краевой отсчёт
00207   *
00208   * Гауссова экстраполяция (E2): MAE = 0.148
00209   *   → в 2-3 раза лучше любого стандартного метода!
00210   */
00211  if (discr_is_monotonic(A1, A2, A3)) {
00212    ret = discr3_extrap_gauss(A1, A2, A3, x1, x2, x3, xe);
00213    return (ret == EXIT_SUCCESS) ? 2 : 3;
00214    /* 2 = E2 сработала, 3 = E2 не смогла (нули, вырожденность) */
00215  }
00216
00217  /*
00218   * Ветка 2: НОРМАЛЬНЫЕ данные (пик внутри сетки)
00219   *
00220   * EA (экспоненциальный): MAE = 0.005 в нормальной зоне
00221   *   Лучший метод! Использует log-параболу, точнее QA.
00222   *
00223   * Если EA вернёт FAILURE (вогнутость, нули), переходим на QA.
00224   * QA: MAE = 0.006 — почти так же точен, но без ограничений
00225   *     на знак амплитуд и выпуклость.
00226   */
00227  ret = discr3ea(A1, A2, A3, x1, x2, x3, xe);
00228
00229  if (ret == EXIT_SUCCESS) {
00230    return 0;  /* EA справился — лучший результат */
00231  }
00232
00233  /*
00234   * Ветка 3: FALLBACK на QA
00235   *
00236   * EA вернул FAILURE. Причины:
00237   *   - Вогнутость (минимум в центре вместо максимума)
00238   *   - Нулевые или отрицательные амплитуды
00239   *   - Числовая вырожденность
00240   *
00241   * QA не требует положительных амплитуд и не проверяет
00242   * выпуклость — работает всегда (кроме совсем вырожденных случаев).
00243   */
00244  *xe = discr3qa(A1, A2, A3, x1, x2, x3);
00245  return 1;  /* QA fallback */
00246 }
```

5.25.2.3 discr3_extrap_gauss()

```
int discr3_extrap_gauss (
            double A1,
            double A2,
```

double A3,
double x1,
double x2,
double x3,
double * xe)

Гауссова экстраполяция (E2) для монотонных случаев.
Алгоритм — как discr3ea, но без проверки выпуклости и с расширенным диапазоном допустимого вылета (2× вместо 0.5× от размера сетки).
Логарифмирует амплитуды, аппроксимирует параболой в log-пространстве, находит вершину. Для монотонных данных вершина за пределами сетки — это ожидаемое и полезное поведение.
Из исследования: E2 MAE = 0.148 для монотонных случаев, что в 2.2 раза лучше E1 (парабола, MAE = 0.324) и в 3.4 раза лучше E3 (градиент, MAE = 0.500).

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
A3	амплитуда третьей точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты (экстраполированная)

Возвращает

0 — успех, 1 — ошибка (нулевые амплитуды, все равны)

См. определение в файле discr_auto.c строка 72

```
00075 {
00076  ax_pair_t pts[3];
00077  double a1, a2, a3, f1, f2, f3;
00078  double z1, z2, z3;
00079  double a_coef, b_coef;
00080  double span;
00081
00082  /*
00083   * Шаг 1. Проверка: все амплитуды > 0
00084   * (нужно для логарифмирования)
00085   */
00086  if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
00087    *xe = x2;
00088    return EXIT_FAILURE;
00089  }
00090
00091  /* Все амплитуды одинаковы → невозможно определить направление */
00092  if (fabs(A1 - A2) < DBL_EPSILON && fabs(A2 - A3) < DBL_EPSILON) {
00093    *xe = x2;
00094    return EXIT_FAILURE;
00095  }
00096
00097  /*
00098   * Шаг 2. Сортировка по координатам (x возрастает)
00099   * Это нужно чтобы формулы работали корректно
00100   * независимо от порядка входных аргументов.
00101   */
00102  pts[0].a = A1; pts[0].x = x1;
00103  pts[1].a = A2; pts[1].x = x2;
00104  pts[2].a = A3; pts[2].x = x3;
00105  qsort(pts, 3, sizeof(ax_pair_t), cmp_by_x);
00106
00107  a1 = pts[0].a; f1 = pts[0].x;
00108  a2 = pts[1].a; f2 = pts[1].x;
00109  a3 = pts[2].a; f3 = pts[2].x;
00110
00111  /*
00112   * Шаг 3. НЕ проверяем выпуклость!
00113   *
00114   * Это ГЛАВНОЕ отличие от discr3ea:
00115   * - discr3ea проверяет что a2 > a1 и a2 > a3 (перегиб)
00116   *   и возвращает FAILURE для монотонных данных
00117   * - Мы РАЗРЕШАЕМ монотонные данные, потому что
```

```
 *   вершина параболы в log-пространстве покажет
 *   КУДА и НАСКОЛЬКО далеко экстраполировать пик.
 */

/*
 * Шаг 4. Логарифмирование
 *
 * Гауссиан y = A*exp(-a*(x-x0)^2) после логарифмирования
 * становится параболой: ln(y) = ln(A) - a*(x-x0)^2
 *
 * Парабола в log-пространстве → вершина = x0 (положение пика).
 */
z1 = log(a1);
z2 = log(a2);
z3 = log(a3);

/*
 * Шаг 5. Коэффициенты параболы
 *
 * Парабола z(x) через 3 точки (f1,z1), (f2,z2), (f3,z3):
 *   a_coef = z1*(f2²-f3²) + z2*(f3²-f1²) + z3*(f1²-f2²)
 *   b_coef = z1*(f2-f3)   + z2*(f3-f1)   + z3*(f1-f2)
 *
 * Вершина: xe = 0.5 * a_coef / b_coef
 */
a_coef = z1 * (f2 * f2 - f3 * f3)
       + z2 * (f3 * f3 - f1 * f1)
       + z3 * (f1 * f1 - f2 * f2);

b_coef = z1 * (f2 - f3)
       + z2 * (f3 - f1)
       + z3 * (f1 - f2);

/* b_coef ≈ 0 → парабола вырожденная, направление неопределено */
if (fabs(b_coef) < DBL_EPSILON) {
  /* Экстраполируем в сторону наибольшей амплитуды */
  if (a3 > a1)
    *xe = f3 + 0.5 * (f3 - f1);
  else
    *xe = f1 - 0.5 * (f3 - f1);
  return EXIT_FAILURE;
}

/* Вершина параболы в log-пространстве = оценка положения пика */
*xe = 0.5 * a_coef / b_coef;

/*
 * Шаг 6. Ограничение вылета (clipping)
 *
 * Расширенный диапазон: ±2 × размер_сетки (вместо ±0.5× в discr3ea).
 * Это позволяет экстраполировать дальше, но не бесконечно далеко.
 *
 * Почему 2×? Из исследования: при вылете > 2× шага ошибка
 * экстраполяции уже не уменьшается, а растёт.
 */
span = f3 - f1;  /* = 2 × шаг_сетки для симметричной сетки */

if (*xe > f3 + span) {
  *xe = f3 + span;
  return EXIT_FAILURE;
}
if (*xe < f1 - span) {
  *xe = f1 - span;
  return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}
```

5.25.2.4 discr_is_monotonic()

```
int discr_is_monotonic (
            double A1,
            double A2,
            double A3 )
```

Проверка монотонности трёх отсчётов.

Монотонные данные = нет локального максимума среди 3 точек. Это означает, что пик за пределами сетки.

Аргументы

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки

Возвращает

1 — монотонные (пик за сеткой), 0 — есть перегиб (пик внутри)

См. определение в файле discr_auto.c строка 35

```
00036 {
00037   /*
00038    * Монотонные = все убывают ИЛИ все возрастают.
00039    * Это значит, что среди A1, A2, A3 нет локального максимума,
00040    * т.е. пик ДН находится за пределами сетки из 3 отсчётов.
00041    *
00042    * Примеры:
00043    *   A1=0.8, A2=0.6, A3=0.3 → убывают → монотонные (пик слева)
00044    *   A1=0.3, A2=0.6, A3=0.8 → возрастают → монотонные (пик справа)
00045    *   A1=0.6, A2=0.9, A3=0.7 → перегиб → НЕ монотонные (пик внутри)
00046    */
00047   return (A1 >= A2 && A2 >= A3) ||  /* убывают (или равны) */
00048          (A1 <= A2 && A2 <= A3);     /* возрастают (или равны) */
00049 }
```

5.26 discr_auto.c

См. документацию.

```
00001
00023 #include "discr_auto.h"
00024 #include "discrea.h"
00025 #include "discrqa.h"
00026
00027 #include <math.h>
00028 #include <float.h>
00029 #include <stdlib.h>
00030
00031 /* ========================================================================== */
00032 /*  discr_is_monotonic                                                        */
00033 /* ========================================================================== */
00034
00035 int discr_is_monotonic(double A1, double A2, double A3)
00036 {
00037   /*
00038    * Монотонные = все убывают ИЛИ все возрастают.
00039    * Это значит, что среди A1, A2, A3 нет локального максимума,
00040    * т.е. пик ДН находится за пределами сетки из 3 отсчётов.
00041    *
00042    * Примеры:
00043    *   A1=0.8, A2=0.6, A3=0.3 → убывают → монотонные (пик слева)
00044    *   A1=0.3, A2=0.6, A3=0.8 → возрастают → монотонные (пик справа)
00045    *   A1=0.6, A2=0.9, A3=0.7 → перегиб → НЕ монотонные (пик внутри)
00046    */
00047   return (A1 >= A2 && A2 >= A3) ||  /* убывают (или равны) */
00048          (A1 <= A2 && A2 <= A3);     /* возрастают (или равны) */
00049 }
00050
00051 /* ========================================================================== */
00052 /*  discr3_extrap_gauss — Гауссова экстраполяция (метод E2)                   */
00053 /* ========================================================================== */
00054
00058 typedef struct {
00059   double a;
00060   double x;
00061 } ax_pair_t;
00062
00063 static int cmp_by_x(const void *p1, const void *p2)
00064 {
00065   const ax_pair_t *a = (const ax_pair_t *)p1;
00066   const ax_pair_t *b = (const ax_pair_t *)p2;
00067   if (a->x < b->x) return -1;
00068   if (a->x > b->x) return  1;
00069   return 0;
```

```c
}

int discr3_extrap_gauss(double A1, double A2, double A3,
                        double x1, double x2, double x3,
                        double *xe)
{
  ax_pair_t pts[3];
  double a1, a2, a3, f1, f2, f3;
  double z1, z2, z3;
  double a_coef, b_coef;
  double span;

  /*
   * Шаг 1. Проверка: все амплитуды > 0
   * (нужно для логарифмирования)
   */
  if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
    *xe = x2;
    return EXIT_FAILURE;
  }

  /* Все амплитуды одинаковы → невозможно определить направление */
  if (fabs(A1 - A2) < DBL_EPSILON && fabs(A2 - A3) < DBL_EPSILON) {
    *xe = x2;
    return EXIT_FAILURE;
  }

  /*
   * Шаг 2. Сортировка по координатам (x возрастает)
   * Это нужно чтобы формулы работали корректно
   * независимо от порядка входных аргументов.
   */
  pts[0].a = A1; pts[0].x = x1;
  pts[1].a = A2; pts[1].x = x2;
  pts[2].a = A3; pts[2].x = x3;
  qsort(pts, 3, sizeof(ax_pair_t), cmp_by_x);

  a1 = pts[0].a; f1 = pts[0].x;
  a2 = pts[1].a; f2 = pts[1].x;
  a3 = pts[2].a; f3 = pts[2].x;

  /*
   * Шаг 3. НЕ проверяем выпуклость!
   *
   * Это ГЛАВНОЕ отличие от discr3ea:
   * - discr3ea проверяет что a2 > a1 и a2 > a3 (перегиб)
   *   и возвращает FAILURE для монотонных данных
   * - Мы РАЗРЕШАЕМ монотонные данные, потому что
   *   вершина параболы в log-пространстве покажет
   *   КУДА и НАСКОЛЬКО далеко экстраполировать пик.
   */

  /*
   * Шаг 4. Логарифмирование
   *
   * Гауссиан y = A*exp(-a*(x-x0)^2) после логарифмирования
   * становится параболой: ln(y) = ln(A) - a*(x-x0)^2
   *
   * Парабола в log-пространстве → вершина = x0 (положение пика).
   */
  z1 = log(a1);
  z2 = log(a2);
  z3 = log(a3);

  /*
   * Шаг 5. Коэффициенты параболы
   *
   * Парабола z(x) через 3 точки (f1,z1), (f2,z2), (f3,z3):
   *   a_coef = z1*(f2^2-f3^2) + z2*(f3^2-f1^2) + z3*(f1^2-f2^2)
   *   b_coef = z1*(f2-f3)   + z2*(f3-f1)     + z3*(f1-f2)
   *
   * Вершина: xe = 0.5 * a_coef / b_coef
   */
  a_coef = z1 * (f2 * f2 - f3 * f3)
         + z2 * (f3 * f3 - f1 * f1)
         + z3 * (f1 * f1 - f2 * f2);

  b_coef = z1 * (f2 - f3)
         + z2 * (f3 - f1)
         + z3 * (f1 - f2);

  /* b_coef ≈ 0 → парабола вырожденная, направление неопределено */
  if (fabs(b_coef) < DBL_EPSILON) {
    /* Экстраполируем в сторону наибольшей амплитуды */
    if (a3 > a1)
      *xe = f3 + 0.5 * (f3 - f1);
    else
```

```
    *xe = f1 - 0.5 * (f3 - f1);
  return EXIT_FAILURE;
}

/* Вершина параболы в log-пространстве = оценка положения пика */
*xe = 0.5 * a_coef / b_coef;

/*
 * Шаг 6. Ограничение вылета (clipping)
 *
 * Расширенный диапазон: ±2 × размер_сетки (вместо ±0.5× в discr3ea).
 * Это позволяет экстраполировать дальше, но не бесконечно далеко.
 *
 * Почему 2×? Из исследования: при вылете > 2× шага ошибка
 * экстраполяции уже не уменьшается, а растёт.
 */
span = f3 - f1;  /* = 2 × шаг_сетки для симметричной сетки */

if (*xe > f3 + span) {
  *xe = f3 + span;
  return EXIT_FAILURE;
}
if (*xe < f1 - span) {
  *xe = f1 - span;
  return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}

/* ========================================================================== */
/*  discr3_auto — автоматический дискриминатор                              */
/* ========================================================================== */

int discr3_auto(double A1, double A2, double A3,
                double x1, double x2, double x3,
                double *xe)
{
  int ret;

  /*
   * Ветка 1: МОНОТОННЫЕ данные (пик за сеткой)
   *
   * Когда A1 ≥ A2 ≥ A3 или A1 ≤ A2 ≤ A3, среди 3 отсчётов
   * нет перегиба — пик ДН находится за пределами сетки.
   *
   * Стандартные дискриминаторы в этом случае:
   *   CG: MAE = 0.46 (почти случайная оценка)
   *   QA: MAE = 0.32, MaxErr = 1.23 (парабола улетает!)
   *   EA: возвращает FAILURE, ставит xe = краевой отсчёт
   *
   * Гауссова экстраполяция (E2): MAE = 0.148
   *   → в 2-3 раза лучше любого стандартного метода!
   */
  if (discr_is_monotonic(A1, A2, A3)) {
    ret = discr3_extrap_gauss(A1, A2, A3, x1, x2, x3, xe);
    return (ret == EXIT_SUCCESS) ? 2 : 3;
    /* 2 = E2 сработала, 3 = E2 не смогла (нули, вырожденность) */
  }

  /*
   * Ветка 2: НОРМАЛЬНЫЕ данные (пик внутри сетки)
   *
   * EA (экспоненциальный): MAE = 0.005 в нормальной зоне
   *   Лучший метод! Использует log-параболу, точнее QA.
   *
   * Если EA вернёт FAILURE (вогнутость, нули), переходим на QA.
   * QA: MAE = 0.006 — почти так же точен, но без ограничений
   *     на знак амплитуд и выпуклость.
   */
  ret = discr3ea(A1, A2, A3, x1, x2, x3, xe);

  if (ret == EXIT_SUCCESS) {
    return 0;  /* EA справился — лучший результат */
  }

  /*
   * Ветка 3: FALLBACK на QA
   *
   * EA вернул FAILURE. Причины:
   *   - Вогнутость (минимум в центре вместо максимума)
   *   - Нулевые или отрицательные амплитуды
   *   - Числовая вырожденность
   *
   * QA не требует положительных амплитуд и не проверяет
   * выпуклость — работает всегда (кроме совсем вырожденных случаев).
   */
```

```
00244   *xe = discr3qa(A1, A2, A3, x1, x2, x3);
00245   return 1;  /* QA fallback */
00246 }
```

5.27 Файл discr_common.c

КППО / ТЗК. Обёртки для дискриминаторных оценок.
#include <stdio.h>
#include <math.h>
#include <float.h>
#include "discr_common.h"
#include "discrcg.h"
#include "discrea.h"
#include "discrqa.h"

Макросы

- #define _USE_MATH_DEFINES

Функции

- int discr2 (double maz[2][2], double mel[2][2], double mval[2][2], double *az, double *el, double *val)

  Двухточечный дискриминатор (обёртка), центр тяжести.
- int discr3 (double **maz, double **mel, double **mval, int ndnx, int ndny, DISCRTYPE discrtype, double dx, double lambda, double *az, double *el, double *val)

  Трехточечный дискриминатор (обёртка).
- int checkuse3d (double **maz, double **mel, double **mval, int ndnx, int ndny)

  Проверка на возможность использования 3-точечного дискриминатора.
- int discr3_ (double maz[3][3], double mel[3][3], double mval[3][3], DISCRTYPE discrtype, double dx, double lambda, double *az, double *el, double *val)

  Адаптер discr3 для статических массивов 3x3.
- int conv3to2 (double maz3[3][3], double mel3[3][3], double mval3[3][3], double maz2[2][2], double mel2[2][2], double mval2[2][2])

  Выделение подматрицы 2x2 из 3x3 для перехода на 2-точечный дискриминатор.

5.27.1 Подробное описание

КППО / ТЗК. Обёртки для дискриминаторных оценок.

Автор

Керский Е.В.

Дата

06.07.2017

Рефакторинг: Кодо, 2026-03-28 Изменения:

- ERR-001: реализованы discr2() и conv3to2()
- ERR-003: discr3() больше НЕ модифицирует входные maz/mel (работает с копиями)
- ERR-004: FLT_EPSILON → DBL_EPSILON
- ERR-006: добавлена поддержка DT_CG в discr3()
- STYLE: единые 2-пробельные отступы

См. определение в файле discr_common.c

5.27.2 Макросы

5.27.2.1 _USE_MATH_DEFINES

#define _USE_MATH_DEFINES
См. определение в файле discr_common.c строка 17

5.27.3 Функции

5.27.3.1 checkuse3d()

```
int checkuse3d (
            double ** maz,
            double ** mel,
            double ** mval,
            int ndnx,
            int ndny )
```

Проверка на возможность использования 3-точечного дискриминатора.
Проверяет что все амплитуды положительны.

Аргументы

maz	[in] массив азимутальных координат (не используется, для совместимости)
mel	[in] массив угломестных координат (не используется, для совместимости)
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y

Возвращает

1 — можно использовать, 0 — нельзя

См. определение в файле discr_common.c строка 192

```
00193 {
00194   int i, j;
00195   (void)maz;
00196   (void)mel;
00197
00198   for (i = 0; i < ndnx; i++)
00199     for (j = 0; j < ndny; j++)
00200       if (mval[i][j] < DBL_EPSILON)  /* ERR-004 fix: was FLT_EPSILON */
00201         return 0;
00202   return 1;
00203 }
```

5.27.3.2 conv3to2()

```
int conv3to2 (
            double maz3[3][3],
            double mel3[3][3],
            double mval3[3][3],
            double maz2[2][2],
            double mel2[2][2],
            double mval2[2][2] )
```

Выделение подматрицы 2x2 из 3x3 для перехода на 2-точечный дискриминатор.
Выбирает квадрант 2x2 с максимальной суммарной амплитудой. Используется когда 3-точечный дискриминатор невозможен (checkuse3d вернул 0, вогнутость и т.д.).

Аргументы

maz3	[in] азимутальные координаты 3x3
mel3	[in] угломестные координаты 3x3

Аргументы

mval3	[in] амплитуды 3x3
maz2	[out] азимутальные координаты 2x2
mel2	[out] угломестные координаты 2x2
mval2	[out] амплитуды 2x2

Возвращает

0 — успех

См. определение в файле discr_common.c строка 249

```
00251 {
00252   int bestR = 0, bestC = 0;
00253   double bestSum = -1.0;
00254   int r, c, i, j;
00255
00256   /* Перебираем все 4 квадранта 2x2 (смещения: (0,0), (0,1), (1,0), (1,1)) */
00257   for (r = 0; r < 2; r++) {
00258     for (c = 0; c < 2; c++) {
00259       double sum = mval3[r][c]     + mval3[r][c + 1]
00260                  + mval3[r + 1][c] + mval3[r + 1][c + 1];
00261       if (sum > bestSum) {
00262         bestSum = sum;
00263         bestR = r;
00264         bestC = c;
00265       }
00266     }
00267   }
00268
00269   /* Копируем выбранный квадрант */
00270   for (i = 0; i < 2; i++) {
00271     for (j = 0; j < 2; j++) {
00272       maz2[i][j]  = maz3[bestR + i][bestC + j];
00273       mel2[i][j]  = mel3[bestR + i][bestC + j];
00274       mval2[i][j] = mval3[bestR + i][bestC + j];
00275     }
00276   }
00277
00278   return 0;
00279 }
```

5.27.3.3 discr2()

```
int discr2 (
            double maz[2][2],
            double mel[2][2],
            double mval[2][2],
            double * az,
            double * el,
            double * val )
```

Двухточечный дискриминатор (обёртка), центр тяжести.
Принимает матрицы 2x2 координат и амплитуд ДН. Находит строку/столбец с максимумом и оценивает az, el через discr2cg().

Аргументы

maz	[in] массив азимутальных координат 2x2
mel	[in] массив угломестных координат 2x2
mval	[in] массив амплитуд 2x2
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка

См. определение в файле discr_common.c строка 43

```
00045 {
00046   int maxRow = 0, maxCol = 0;
00047   double maxAmpl = -1.0;
00048   int i, j;
00049
00050   /* Проверка: все амплитуды должны быть положительны */
00051   for (i = 0; i < 2; ++i)
00052     for (j = 0; j < 2; ++j)
00053       if (mval[i][j] < DBL_EPSILON) {
00054         *az = 0.0; *el = 0.0; *val = 0.0;
00055         return -1;
00056       }
00057
00058   /* Поиск максимума */
00059   for (i = 0; i < 2; ++i)
00060     for (j = 0; j < 2; ++j)
00061       if (mval[i][j] > maxAmpl) {
00062         maxRow = i;
00063         maxCol = j;
00064         maxAmpl = mval[i][j];
00065       }
00066
00067   /* Оценка по азимуту (по строке maxRow) */
00068   *az = discr2cg(mval[maxRow][0], mval[maxRow][1],
00069                  maz[maxRow][0],  maz[maxRow][1]);
00070
00071   /* Оценка по углу места (по столбцу maxCol) */
00072   *el = discr2cg(mval[0][maxCol], mval[1][maxCol],
00073                  mel[0][maxCol],  mel[1][maxCol]);
00074
00075   *val = maxAmpl;
00076   return 0;
00077 }
```

5.27.3.4 discr3()

```
int discr3 (
            double ** maz,
            double ** mel,
            double ** mval,
            int ndnx,
            int ndny,
            DISCRTYPE discrtype,
            double dx,
            double lambda,
            double * az,
            double * el,
            double * val )
```

Трехточечный дискриминатор (обёртка).
Принимает матрицы ndnx x ndny координат и амплитуд ДН. Поддерживает типы: DT_CG, DT↩_QA, DT_EA.

Предупреждения

Функция НЕ модифицирует входные массивы maz, mel (исправлено в рефакторинге, ERR-003).

Аргументы

maz	[in] массив азимутальных координат
mel	[in] массив угломестных координат
mval	[in] массив амплитуд
ndnx	[in] число ДН по оси X
ndny	[in] число ДН по оси Y

Аргументы

discrtype	[in] тип дискриминатора (DT_CG, DT_QA, DT_EA)
dx	[in] шаг по углу (используется для DT_QA, DT_EA)
lambda	[in] длина волны (используется для DT_QA, DT_EA)
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — нельзя использовать 3-точ., -2 — неизвестный тип

См. определение в файле discr_common.c строка 101

```
00104 {
00105   double aztmp = 0.0, eltmp = 0.0, valaz = 0.0, valel = 0.0;
00106   int maxRow = 0, maxCol = 0;
00107   double maxAmpl = -1.0;
00108   const double c = 2.0 * M_PI * dx / lambda;
00109   int i, j;
00110
00111   /* Проверка на возможность использования 3-точечного дискриминатора */
00112   if (!checkuse3d(maz, mel, mval, ndnx, ndny))
00113     return -1;
00114
00115   /* Поиск строки и столбца с максимальной амплитудой */
00116   for (i = 0; i < ndnx; ++i)
00117     for (j = 0; j < ndny; ++j)
00118       if (mval[i][j] > maxAmpl) {
00119         maxRow = i;
00120         maxCol = j;
00121         maxAmpl = mval[i][j];
00122       }
00123
00124   switch (discrtype) {
00125     case DT_CG:
00126       /* ERR-006 fix: добавлена поддержка центра тяжести */
00127       if (ndny >= 3) {
00128         aztmp = discr3cg(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00129                          maz[maxRow][0],  maz[maxRow][1],  maz[maxRow][2]);
00130       }
00131       if (ndnx >= 3) {
00132         eltmp = discr3cg(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00133                          mel[0][maxCol],  mel[1][maxCol],  mel[2][maxCol]);
00134       }
00135       *val = maxAmpl;
00136       break;
00137
00138     case DT_QA:
00139       /* ERR-003 fix: координаты умножаем на c локально, не модифицируя входные данные */
00140       aztmp = discr3qa(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00141                        maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c);
00142       eltmp = discr3qa(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00143                        mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c);
00144       *val = maxAmpl;
00145       /* Обратное преобразование из масштабированных координат */
00146       *az = aztmp / c;
00147       *el = eltmp / c;
00148       return 0;
00149
00150     case DT_EA:
00151       /* ERR-003 fix: координаты умножаем на c локально */
00152       discr3ea(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00153                maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c, &aztmp);
00154       discr3ea(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00155                mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c, &eltmp);
00156
00157       valaz = discr3eaY(mval[maxRow][0], mval[maxRow][1],
00158                         maz[maxRow][0] * c, maz[maxRow][1] * c, aztmp);
00159       valel = discr3eaY(mval[0][maxCol], mval[1][maxCol],
00160                         mel[0][maxCol] * c, mel[1][maxCol] * c, eltmp);
00161
00162       *val = (valaz > valel) ? valaz : valel;
00163       *val = (maxAmpl > *val) ? maxAmpl : *val;
00164       /* Обратное преобразование */
00165       *az = aztmp / c;
00166       *el = eltmp / c;
00167       return 0;
```

```
00168
00169     default:
00170       *az = 0.0; *el = 0.0; *val = 0.0;
00171       return -2;
00172   }
00173
00174   /* Для DT_CG — координаты не масштабируются */
00175   *az = aztmp;
00176   *el = eltmp;
00177   return 0;
00178 }
```

5.27.3.5 discr3_()

```
int discr3_ (
            double maz[3][3],
            double mel[3][3],
            double mval[3][3],
            DISCRTYPE discrtype,
            double dx,
            double lambda,
            double * az,
            double * el,
            double * val )
```

Адаптер discr3 для статических массивов 3x3.
Адаптер трехточечного дискриминатора для статических массивов 3x3.
Преобразует массивы double[3][3] в double** для вызова discr3().

Аргументы

maz	[in] массив азимутальных координат 3x3
mel	[in] массив угломестных координат 3x3
mval	[in] массив амплитуд 3x3
discrtype	[in] тип дискриминатора
dx	[in] шаг по углу
lambda	[in] длина волны
az	[out] уточнённая координата азимута
el	[out] уточнённая координата угла места
val	[out] уточнённое значение амплитуды

Возвращает

0 — норма, -1 — ошибка, -2 — неизвестный тип

См. определение в файле discr_common.c строка 221

```
00224 {
00225   double *pmaz[3]  = { &maz[0][0],  &maz[1][0],  &maz[2][0] };
00226   double *pmel[3]  = { &mel[0][0],  &mel[1][0],  &mel[2][0] };
00227   double *pmval[3] = { &mval[0][0], &mval[1][0], &mval[2][0] };
00228
00229   return discr3(pmaz, pmel, pmval, 3, 3,
00230                 discrtype, dx, lambda,
00231                 az, el, val);
00232 }
```

5.28 discr_common.c

См. документацию.

```
00001
00016 #ifndef _USE_MATH_DEFINES
00017 #define _USE_MATH_DEFINES
00018 #endif
00019 #include <stdio.h>
```

```c
#include <math.h>
#include <float.h>

#include "discr_common.h"
#include "discrcg.h"
#include "discrea.h"
#include "discrqa.h"

int discr2(double maz[2][2], double mel[2][2], double mval[2][2],
           double *az, double *el, double *val)
{
  int maxRow = 0, maxCol = 0;
  double maxAmpl = -1.0;
  int i, j;

  /* Проверка: все амплитуды должны быть положительны */
  for (i = 0; i < 2; ++i)
    for (j = 0; j < 2; ++j)
      if (mval[i][j] < DBL_EPSILON) {
        *az = 0.0; *el = 0.0; *val = 0.0;
        return -1;
      }

  /* Поиск максимума */
  for (i = 0; i < 2; ++i)
    for (j = 0; j < 2; ++j)
      if (mval[i][j] > maxAmpl) {
        maxRow = i;
        maxCol = j;
        maxAmpl = mval[i][j];
      }

  /* Оценка по азимуту (по строке maxRow) */
  *az = discr2cg(mval[maxRow][0], mval[maxRow][1],
                 maz[maxRow][0],  maz[maxRow][1]);

  /* Оценка по углу места (по столбцу maxCol) */
  *el = discr2cg(mval[0][maxCol], mval[1][maxCol],
                 mel[0][maxCol],  mel[1][maxCol]);

  *val = maxAmpl;
  return 0;
}

int discr3(double **maz, double **mel, double **mval, int ndnx, int ndny,
           DISCRTYPE discrtype, double dx, double lambda,
           double *az, double *el, double *val)
{
  double aztmp = 0.0, eltmp = 0.0, valaz = 0.0, valel = 0.0;
  int maxRow = 0, maxCol = 0;
  double maxAmpl = -1.0;
  const double c = 2.0 * M_PI * dx / lambda;
  int i, j;

  /* Проверка на возможность использования 3-точечного дискриминатора */
  if (!checkuse3d(maz, mel, mval, ndnx, ndny))
    return -1;

  /* Поиск строки и столбца с максимальной амплитудой */
  for (i = 0; i < ndnx; ++i)
    for (j = 0; j < ndny; ++j)
      if (mval[i][j] > maxAmpl) {
        maxRow = i;
        maxCol = j;
        maxAmpl = mval[i][j];
      }

  switch (discrtype) {
    case DT_CG:
      /* ERR-006 fix: добавлена поддержка центра тяжести */
      if (ndny >= 3) {
        aztmp = discr3cg(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
                         maz[maxRow][0],  maz[maxRow][1],  maz[maxRow][2]);
      }
      if (ndnx >= 3) {
        eltmp = discr3cg(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
                         mel[0][maxCol],  mel[1][maxCol],  mel[2][maxCol]);
      }
      *val = maxAmpl;
      break;

    case DT_QA:
      /* ERR-003 fix: координаты умножаем на c локально, не модифицируя входные данные */
      aztmp = discr3qa(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
                       maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c);
      eltmp = discr3qa(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
                       mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c);
```

```
00144       *val = maxAmpl;
00145       /* Обратное преобразование из масштабированных координат */
00146       *az = aztmp / c;
00147       *el = eltmp / c;
00148       return 0;
00149
00150     case DT_EA:
00151       /* ERR-003 fix: координаты умножаем на c локально */
00152       discr3ea(mval[maxRow][0], mval[maxRow][1], mval[maxRow][2],
00153               maz[maxRow][0] * c, maz[maxRow][1] * c, maz[maxRow][2] * c, &aztmp);
00154       discr3ea(mval[0][maxCol], mval[1][maxCol], mval[2][maxCol],
00155               mel[0][maxCol] * c, mel[1][maxCol] * c, mel[2][maxCol] * c, &eltmp);
00156
00157       valaz = discr3eaY(mval[maxRow][0], mval[maxRow][1],
00158                       maz[maxRow][0] * c, maz[maxRow][1] * c, aztmp);
00159       valel = discr3eaY(mval[0][maxCol], mval[1][maxCol],
00160                       mel[0][maxCol] * c, mel[1][maxCol] * c, eltmp);
00161
00162       *val = (valaz > valel) ? valaz : valel;
00163       *val = (maxAmpl > *val) ? maxAmpl : *val;
00164       /* Обратное преобразование */
00165       *az = aztmp / c;
00166       *el = eltmp / c;
00167       return 0;
00168
00169     default:
00170       *az = 0.0; *el = 0.0; *val = 0.0;
00171       return -2;
00172   }
00173
00174   /* Для DT_CG — координаты не масштабируются */
00175   *az = aztmp;
00176   *el = eltmp;
00177   return 0;
00178 }
00179
00192 int checkuse3d(double **maz, double **mel, double **mval, int ndnx, int ndny)
00193 {
00194   int i, j;
00195   (void)maz;
00196   (void)mel;
00197
00198   for (i = 0; i < ndnx; i++)
00199     for (j = 0; j < ndny; j++)
00200       if (mval[i][j] < DBL_EPSILON)  /* ERR-004 fix: was FLT_EPSILON */
00201         return 0;
00202   return 1;
00203 }
00204
00221 int discr3_(double maz[3][3], double mel[3][3], double mval[3][3],
00222             DISCRTYPE discrtype, double dx, double lambda,
00223             double *az, double *el, double *val)
00224 {
00225   double *pmaz[3]  = { &maz[0][0],  &maz[1][0],  &maz[2][0] };
00226   double *pmel[3]  = { &mel[0][0],  &mel[1][0],  &mel[2][0] };
00227   double *pmval[3] = { &mval[0][0], &mval[1][0], &mval[2][0] };
00228
00229   return discr3(pmaz, pmel, pmval, 3, 3,
00230                 discrtype, dx, lambda,
00231                 az, el, val);
00232 }
00233
00249 int conv3to2(double maz3[3][3], double mel3[3][3], double mval3[3][3],
00250             double maz2[2][2], double mel2[2][2], double mval2[2][2])
00251 {
00252   int bestR = 0, bestC = 0;
00253   double bestSum = -1.0;
00254   int r, c, i, j;
00255
00256   /* Перебираем все 4 квадранта 2x2 (смещения: (0,0), (0,1), (1,0), (1,1)) */
00257   for (r = 0; r < 2; r++) {
00258     for (c = 0; c < 2; c++) {
00259       double sum = mval3[r][c]     + mval3[r][c + 1]
00260                  + mval3[r + 1][c] + mval3[r + 1][c + 1];
00261       if (sum > bestSum) {
00262         bestSum = sum;
00263         bestR = r;
00264         bestC = c;
00265       }
00266     }
00267   }
00268
00269   /* Копируем выбранный квадрант */
00270   for (i = 0; i < 2; i++) {
00271     for (j = 0; j < 2; j++) {
00272       maz2[i][j]  = maz3[bestR + i][bestC + j];
00273       mel2[i][j]  = mel3[bestR + i][bestC + j];
```

```
00274       mval2[i][j] = mval3[bestR + i][bestC + j];
00275     }
00276   }
00277
00278   return 0;
00279 }
```

5.29 Файл discrcg.c

КППО / ТЗК. Дискриминатор — центр тяжести (Center of Gravity).
#include "discrcg.h"
#include <math.h>
#include <float.h>

Функции

- double discr2cg (double A1, double A2, double x1, double x2)

  Двухточечный дискриминатор, центр тяжести.
- double discr3cg (double A1, double A2, double A3, double x1, double x2, double x3)

  Трехточечный дискриминатор, центр тяжести.
- double discrncg (const double *A, const double *x, unsigned int N)

  Дискриминатор для произвольного числа точек, центр тяжести.

5.29.1 Подробное описание

КППО / ТЗК. Дискриминатор — центр тяжести (Center of Gravity).

Автор

Федоров С.А.

Дата

07.05.2010

Рефакторинг: Кодо, 2026-03-28 Изменения:

- ERR-005: добавлена защита от деления на 0 (A1+A2=0, As=0)
- STYLE: единые 2-пробельные отступы, пробелы вокруг операторов

См. определение в файле discrcg.c

5.29.2 Функции

5.29.2.1 discr2cg()

```
double discr2cg (
            double A1,
            double A2,
            double x1,
            double x2 )
```

Двухточечный дискриминатор, центр тяжести.
Формула: xe = (A1*x1 + A2*x2) / (A1 + A2)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты; при A1+A2=0 возвращает середину (x1+x2)/2

См. определение в файле discrcg.c строка 28

```
00030 {
00031   double sum = A1 + A2;
00032   if (fabs(sum) < DBL_EPSILON)
00033     return (x1 + x2) * 0.5;
00034   return (A1 * x1 + A2 * x2) / sum;
00035 }
```

5.29.2.2 discr3cg()

```
double discr3cg (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3 )
```

Трехточечный дискриминатор, центр тяжести.
Формула: xe = (A1∗x1 + A2∗x2 + A3∗x3) / (A1 + A2 + A3)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты; при сумме=0 возвращает x2 (центральная точка)

См. определение в файле discrcg.c строка 50

```
00052 {
00053   double sum = A1 + A2 + A3;
00054   if (fabs(sum) < DBL_EPSILON)
00055     return x2;
00056   return (A1 * x1 + A2 * x2 + A3 * x3) / sum;
00057 }
```

5.29.2.3 discrncg()

```
double discrncg (
            const double * A,
            const double * x,
            unsigned int N )
```

Дискриминатор для произвольного числа точек, центр тяжести.
Формула: xe = sum(A[i]∗x[i]) / sum(A[i])

Аргументы

A	массив амплитуд точек (N элементов)
x	массив координат точек (N элементов)
N	число точек

Возвращает

оценка координаты; при sum(A)=0 возвращает x[N/2]

См. определение в файле discrcg.c строка 69

```
00070 {
00071   unsigned int n;
00072   double Axs = 0.0, As = 0.0;
00073
00074   for (n = 0; n < N; n++) {
00075     As += A[n];
00076     Axs += A[n] * x[n];
00077   }
00078
00079   if (fabs(As) < DBL_EPSILON) {
00080     if (N > 0)
00081       return x[N / 2];
00082     return 0.0;
00083   }
00084   return Axs / As;
00085 }
```

5.30 discrcg.c

См. документацию.

```
00001
00013 #include "discrcg.h"
00014 #include <math.h>
00015 #include <float.h>
00016
00028 double discr2cg(double A1, double A2,
00029                double x1, double x2)
00030 {
00031   double sum = A1 + A2;
00032   if (fabs(sum) < DBL_EPSILON)
00033     return (x1 + x2) * 0.5;
00034   return (A1 * x1 + A2 * x2) / sum;
00035 }
00036
00050 double discr3cg(double A1, double A2, double A3,
00051                double x1, double x2, double x3)
00052 {
00053   double sum = A1 + A2 + A3;
00054   if (fabs(sum) < DBL_EPSILON)
00055     return x2;
00056   return (A1 * x1 + A2 * x2 + A3 * x3) / sum;
00057 }
00058
00069 double discrncg(const double *A, const double *x, unsigned int N)
00070 {
00071   unsigned int n;
00072   double Axs = 0.0, As = 0.0;
00073
00074   for (n = 0; n < N; n++) {
00075     As += A[n];
00076     Axs += A[n] * x[n];
00077   }
00078
00079   if (fabs(As) < DBL_EPSILON) {
00080     if (N > 0)
00081       return x[N / 2];
00082     return 0.0;
00083   }
00084   return Axs / As;
00085 }
```

5.31 Файл discrea.c

КППО / ТЗК. Дискриминатор — экспоненциальная аппроксимация (Exponential).

#include <float.h>
#include <math.h>
#include <stdlib.h>
#include "discrea.h"

Структуры данных

- struct axstr_t

  Структура для сортировки пар (амплитуда, координата).

Функции

- static int compare_by_x (const void *p1, const void *p2)

  Сортировка пар (амплитуда, координата) по возрастанию координаты.
- int discr3ea (double A1, double A2, double A3, double x1, double x2, double x3, double *xe)

  Трехточечный дискриминатор, экспоненциальная аппроксимация.
- double discr3eaY (double A1, double A2, double x1, double x2, double xe)

  Уточнение амплитуды по экспоненциальной аппроксимации.

5.31.1 Подробное описание

КППО / ТЗК. Дискриминатор — экспоненциальная аппроксимация (Exponential).

Автор

Добродумов А.Б.

Дата

15.04.2012

Аппроксимация отсчётов ДН функцией y = A_max * exp(-a*(x - x0)^2) для нахождения координаты максимума x0.
Рефакторинг: Кодо, 2026-03-28 Изменения:

- ERR-002: static ax[3] перенесена в локальную переменную (thread-safety)
- STYLE: единые 2-пробельные отступы

См. определение в файле discrea.c

5.31.2 Функции

5.31.2.1 compare_by_x()

```
static int compare_by_x (
            const void * p1,
            const void * p2 )   [static]
```

Сортировка пар (амплитуда, координата) по возрастанию координаты.
См. определение в файле discrea.c строка 176

```
00177 {
00178   const axstr_t *ax1 = (const axstr_t *)p1;
00179   const axstr_t *ax2 = (const axstr_t *)p2;
00180
00181   if (ax1->x < ax2->x)
00182     return -1;
00183   else if (ax1->x > ax2->x)
00184     return 1;
00185   return 0;
00186 }
```

5.31.2.2 discr3ea()

```
int discr3ea (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
```

double x3,
double * xe)

Трехточечный дискриминатор, экспоненциальная аппроксимация.
Трехточечный дискриминатор, экспоненциальная аппроксимация (Exponential).
Алгоритм:

1. Проверка входных данных (ненулевые амплитуды, не все равны)
2. Сортировка по координате (x должна возрастать)
3. Проверка на выпуклость (максимум должен быть в центре)
4. Логарифмирование: z = ln(A), аппроксимация параболой z(x)
5. Вершина параболы: xe = 0.5 * a / b
6. Ограничение на вылет за пределы зондирования

Аргументы

A1	амплитуда первой точки (должна быть > 0)
A2	амплитуда второй точки (должна быть > 0)
A3	амплитуда третьей точки (должна быть > 0)
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки
xe	[out] оценка координаты

Возвращает

EXIT_SUCCESS (0) при успехе, EXIT_FAILURE (1) при ошибке (при ошибке *xe содержит приближённое значение)

См. определение в файле discrea.c строка 50

```
00052 {
00053   double a = 0.0, b = 0.0;
00054   double z1, z2, z3;
00055   double f1, f2, f3;
00056   double a1, a2, a3, a12, a23, a13;
00057   /* ERR-002 fix: локальная переменная вместо static */
00058   axstr_t ax[3];
00059
00060   /* Проверка: все амплитуды должны быть положительны */
00061   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
00062     *xe = x2;
00063     return EXIT_FAILURE;
00064   }
00065
00066   /* Проверка: не все амплитуды равны */
00067   if ((fabs(A1 - A2) < DBL_EPSILON) && (fabs(A2 - A3) < DBL_EPSILON)) {
00068     *xe = x2;
00069     return EXIT_FAILURE;
00070   }
00071
00072   /* Сортировка по координате (x должна возрастать) */
00073   ax[0].a = A1; ax[0].x = x1;
00074   ax[1].a = A2; ax[1].x = x2;
00075   ax[2].a = A3; ax[2].x = x3;
00076   qsort(ax, 3, sizeof(axstr_t), compare_by_x);
00077
00078   a1 = ax[0].a; a2 = ax[1].a; a3 = ax[2].a;
00079   f1 = ax[0].x; f2 = ax[1].x; f3 = ax[2].x;
00080
00081   /* Проверка на выпуклость: максимум должен быть в центре */
00082   a12 = a1 - a2;
00083   a23 = a2 - a3;
00084   a13 = a1 - a3;
00085
00086   /* Вогнутость: максимум справа (например 2, 4, 8) */
00087   if ((a12 < 0 && a23 < 0 && a12 > a23) ||
```

```
    (a12 > 0 && a23 < 0 && a13 < 0)) {
  *xe = x3;
  return EXIT_FAILURE;
}
/* Вогнутость: максимум слева (например 8, 4, 2) */
if ((a12 > 0 && a23 > 0 && a12 > a23) ||
    (a12 > 0 && a23 < 0 && a13 > 0)) {
  *xe = x1;
  return EXIT_FAILURE;
}

/* Логарифмирование */
z1 = log(a1);
z2 = log(a2);
z3 = log(a3);

/* Коэффициенты параболы z(x) = a + b*x + ... */
a = z1 * (f2 * f2 - f3 * f3) + z2 * (f3 * f3 - f1 * f1) + z3 * (f1 * f1 - f2 * f2);
b = z1 * (f2 - f3) + z2 * (f3 - f1) + z3 * (f1 - f2);

if (fabs(b) < DBL_EPSILON) {
  if (a * b >= 0) {
    *xe = f3 + 0.5 * (f3 - f1);
    return EXIT_FAILURE;
  } else {
    *xe = f1 - 0.5 * (f3 - f1);
    return EXIT_FAILURE;
  }
}

/* Вершина параболы */
*xe = 0.5 * a / b;

/* Ограничение на вылет за пределы зондирования */
if (*xe > f3 + 0.5 * (f3 - f1)) {
  *xe = f3 + 0.5 * (f3 - f1);
  return EXIT_FAILURE;
}
if (*xe < f1 - 0.5 * (f3 - f1)) {
  *xe = f1 - 0.5 * (f3 - f1);
  return EXIT_FAILURE;
}

return EXIT_SUCCESS;
}
```

5.31.2.3 discr3eaY()

```
double discr3eaY (
            double A1,
            double A2,
            double x1,
            double x2,
            double xe )
```

Уточнение амплитуды по экспоненциальной аппроксимации.
Формула: ye = A2 ∗ exp(a0 ∗ (x2 - xe)^2)

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки
xe	вычисленная координата (результат discr3ea)

Возвращает

уточнённое значение амплитуды

См. определение в файле discrea.c строка 146

```
{
  double ye, yMax;
  double z1, z2;
```

```
00150   double a0, dxe;
00151
00152   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON)
00153     return 0.0;
00154
00155   yMax = (A1 > A2) ? A1 : A2;
00156
00157   z1 = log(A1);
00158   z2 = log(A2);
00159
00160   a0 = (2.0 * xe * (x1 - x2) - x1 * x1 + x2 * x2);
00161
00162   if (fabs(a0) < DBL_EPSILON)
00163     return yMax;
00164
00165   a0 = (z1 - z2) / a0;
00166
00167   dxe = x2 - xe;
00168   ye = A2 * exp(a0 * dxe * dxe);
00169
00170   return ye;
00171 }
```

5.32 discrea.c

См. документацию.

```
00001
00016 #include <float.h>
00017 #include <math.h>
00018 #include <stdlib.h>
00019 #include "discrea.h"
00020
00022 typedef struct {
00023   double a;
00024   double x;
00025 } axstr_t;
00026
00027 static int compare_by_x(const void *p1, const void *p2);
00028
00050 int discr3ea(double A1, double A2, double A3,
00051              double x1, double x2, double x3, double *xe)
00052 {
00053   double a = 0.0, b = 0.0;
00054   double z1, z2, z3;
00055   double f1, f2, f3;
00056   double a1, a2, a3, a12, a23, a13;
00057   /* ERR-002 fix: локальная переменная вместо static */
00058   axstr_t ax[3];
00059
00060   /* Проверка: все амплитуды должны быть положительны */
00061   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON || A3 < DBL_EPSILON) {
00062     *xe = x2;
00063     return EXIT_FAILURE;
00064   }
00065
00066   /* Проверка: не все амплитуды равны */
00067   if ((fabs(A1 - A2) < DBL_EPSILON) && (fabs(A2 - A3) < DBL_EPSILON)) {
00068     *xe = x2;
00069     return EXIT_FAILURE;
00070   }
00071
00072   /* Сортировка по координате (x должна возрастать) */
00073   ax[0].a = A1; ax[0].x = x1;
00074   ax[1].a = A2; ax[1].x = x2;
00075   ax[2].a = A3; ax[2].x = x3;
00076   qsort(ax, 3, sizeof(axstr_t), compare_by_x);
00077
00078   a1 = ax[0].a; a2 = ax[1].a; a3 = ax[2].a;
00079   f1 = ax[0].x; f2 = ax[1].x; f3 = ax[2].x;
00080
00081   /* Проверка на выпуклость: максимум должен быть в центре */
00082   a12 = a1 - a2;
00083   a23 = a2 - a3;
00084   a13 = a1 - a3;
00085
00086   /* Вогнутость: максимум справа (например 2, 4, 8) */
00087   if ((a12 < 0 && a23 < 0 && a12 > a23) ||
00088       (a12 > 0 && a23 < 0 && a13 < 0)) {
00089     *xe = x3;
00090     return EXIT_FAILURE;
00091   }
00092   /* Вогнутость: максимум слева (например 8, 4, 2) */
00093   if ((a12 > 0 && a23 > 0 && a12 > a23) ||
00094       (a12 > 0 && a23 < 0 && a13 > 0)) {
```

```
00095     *xe = x1;
00096     return EXIT_FAILURE;
00097   }
00098
00099   /* Логарифмирование */
00100   z1 = log(a1);
00101   z2 = log(a2);
00102   z3 = log(a3);
00103
00104   /* Коэффициенты параболы z(x) = a + b*x + ... */
00105   a = z1 * (f2 * f2 - f3 * f3) + z2 * (f3 * f3 - f1 * f1) + z3 * (f1 * f1 - f2 * f2);
00106   b = z1 * (f2 - f3) + z2 * (f3 - f1) + z3 * (f1 - f2);
00107
00108   if (fabs(b) < DBL_EPSILON) {
00109     if (a * b >= 0) {
00110       *xe = f3 + 0.5 * (f3 - f1);
00111       return EXIT_FAILURE;
00112     } else {
00113       *xe = f1 - 0.5 * (f3 - f1);
00114       return EXIT_FAILURE;
00115     }
00116   }
00117
00118   /* Вершина параболы */
00119   *xe = 0.5 * a / b;
00120
00121   /* Ограничение на вылет за пределы зондирования */
00122   if (*xe > f3 + 0.5 * (f3 - f1)) {
00123     *xe = f3 + 0.5 * (f3 - f1);
00124     return EXIT_FAILURE;
00125   }
00126   if (*xe < f1 - 0.5 * (f3 - f1)) {
00127     *xe = f1 - 0.5 * (f3 - f1);
00128     return EXIT_FAILURE;
00129   }
00130
00131   return EXIT_SUCCESS;
00132 }
00133
00146 double discr3eaY(double A1, double A2, double x1, double x2, double xe)
00147 {
00148   double ye, yMax;
00149   double z1, z2;
00150   double a0, dxe;
00151
00152   if (A1 < DBL_EPSILON || A2 < DBL_EPSILON)
00153     return 0.0;
00154
00155   yMax = (A1 > A2) ? A1 : A2;
00156
00157   z1 = log(A1);
00158   z2 = log(A2);
00159
00160   a0 = (2.0 * xe * (x1 - x2) - x1 * x1 + x2 * x2);
00161
00162   if (fabs(a0) < DBL_EPSILON)
00163     return yMax;
00164
00165   a0 = (z1 - z2) / a0;
00166
00167   dxe = x2 - xe;
00168   ye = A2 * exp(a0 * dxe * dxe);
00169
00170   return ye;
00171 }
00172
00176 static int compare_by_x(const void *p1, const void *p2)
00177 {
00178   const axstr_t *ax1 = (const axstr_t *)p1;
00179   const axstr_t *ax2 = (const axstr_t *)p2;
00180
00181   if (ax1->x < ax2->x)
00182     return -1;
00183   else if (ax1->x > ax2->x)
00184     return 1;
00185   return 0;
00186 }
```

5.33 Файл discrqa.c

КППО / ТЗК. Дискриминатор — квадратичная аппроксимация (Quadratic).

#include "discrqa.h"

#include <float.h>

#include <math.h>

Функции

- double discr3qa (double A1, double A2, double A3, double x1, double x2, double x3)

  Трехточечный дискриминатор, аппроксимация квадратичной зависимостью.

5.33.1 Подробное описание

КППО / ТЗК. Дискриминатор — квадратичная аппроксимация (Quadratic).

Автор

Федоров С.А.

Дата

07.05.2010

Рефакторинг: Кодо, 2026-03-28 Изменения:

- ERR-004: FLT_EPSILON заменён на DBL_EPSILON для double
- STYLE-001: табуляции заменены на 2 пробела
- STYLE-003: добавлены пробелы вокруг операторов

См. определение в файле discrqa.c

5.33.2 Функции

5.33.2.1 discr3qa()

```
double discr3qa (
            double A1,
            double A2,
            double A3,
            double x1,
            double x2,
            double x3 )
```

Трехточечный дискриминатор, аппроксимация квадратичной зависимостью.
Трехточечный дискриминатор, аппроксимация квадратичной зависимостью (Quadratic).
Аппроксимирует 3 точки параболой $y = a*x^2 + b*x + c$ и находит вершину.
Формулы: Ao = (A2 - A1) / (A2 - A3) xe = $0.5 * ((Ao-1)*x2^2 - Ao*x3^2 + x1^2) / ((Ao-1)*x2 - Ao*x3 + x1)$
Граничные случаи:

- A2 = A3: если A1 = A2 → x2; если A1 > A2 → x1; иначе → (x2+x3)/2
- A1 = A2: если A3 > A2 → x3; иначе → (x1+x2)/2

Аргументы

A1	амплитуда первой точки
A2	амплитуда второй точки
A3	амплитуда третьей точки
x1	координата первой точки
x2	координата второй точки
x3	координата третьей точки

Возвращает

оценка координаты

См. определение в файле discrqa.c строка 39

```
00041 {
00042   if (fabs(A2 - A3) < DBL_EPSILON) {
00043     if (fabs(A2 - A1) < DBL_EPSILON)
00044       return x2;
00045     if (A1 > A2)
00046       return x1;
00047     else
00048       return 0.5 * (x3 + x2);
00049   }
00050
00051   if (fabs(A2 - A1) < DBL_EPSILON) {
00052     if (A3 > A2)
00053       return x3;
00054     else
00055       return 0.5 * (x1 + x2);
00056   }
00057
00058   {
00059     const double Ao = (A2 - A1) / (A2 - A3);
00060     double denom = (Ao - 1.0) * x2 - Ao * x3 + x1;
00061
00062     if (fabs(denom) < DBL_EPSILON)
00063       return x2;
00064
00065     return 0.5 * ((Ao - 1.0) * x2 * x2 - Ao * x3 * x3 + x1 * x1) / denom;
00066   }
00067 }
```

5.34 discrqa.c

См. документацию.

```
00001
00014 #include "discrqa.h"
00015 #include <float.h>
00016 #include <math.h>
00017
00039 double discr3qa(double A1, double A2, double A3,
00040                double x1, double x2, double x3)
00041 {
00042   if (fabs(A2 - A3) < DBL_EPSILON) {
00043     if (fabs(A2 - A1) < DBL_EPSILON)
00044       return x2;
00045     if (A1 > A2)
00046       return x1;
00047     else
00048       return 0.5 * (x3 + x2);
00049   }
00050
00051   if (fabs(A2 - A1) < DBL_EPSILON) {
00052     if (A3 > A2)
00053       return x3;
00054     else
00055       return 0.5 * (x1 + x2);
00056   }
00057
00058   {
00059     const double Ao = (A2 - A1) / (A2 - A3);
00060     double denom = (Ao - 1.0) * x2 - Ao * x3 + x1;
00061
00062     if (fabs(denom) < DBL_EPSILON)
00063       return x2;
00064
00065     return 0.5 * ((Ao - 1.0) * x2 * x2 - Ao * x3 * x3 + x1 * x1) / denom;
00066   }
00067 }
```

5.35 Файл discrsd.c

КППО / ТЗК. Дискриминатор — суммарно-разностный (Sum-Difference).
#include "discrsd.h"
#include <math.h>
#include <float.h>

Функции

- double discr2sd (double c, double A1, double A2, double x1, double x2)

  Двухточечный дискриминатор, суммарно-разностный.

5.35.1 Подробное описание

КППО / ТЗК. Дискриминатор — суммарно-разностный (Sum-Difference).

Автор

Федоров С.А.

Дата

07.05.2010

Рефакторинг: Кодо, 2026-03-28 Изменения:

- ERR-005: добавлена защита от деления на 0 (A2+A1=0)
- STYLE: единые 2-пробельные отступы

См. определение в файле discrsd.c

5.35.2 Функции

5.35.2.1 discr2sd()

```
double discr2sd (
            double c,
            double A1,
            double A2,
            double x1,
            double x2 )
```

Двухточечный дискриминатор, суммарно-разностный.
Двухточечный дискриминатор, суммарно-разностный (Sum-Difference).
Формулы: xc = (x1 + x2) / 2 dx = c ∗ (A2 - A1) / (A2 + A1) xe = xc + dx

Аргументы

c	коэффициент суммарно-разностного дискриминатора
A1	амплитуда первой точки
A2	амплитуда второй точки
x1	координата первой точки
x2	координата второй точки

Возвращает

оценка координаты; при A1+A2=0 возвращает середину (x1+x2)/2

См. определение в файле discrsd.c строка 32

```
00034 {
00035   double xc = (x1 + x2) * 0.5;
00036   double sum = A2 + A1;
00037
00038   if (fabs(sum) < DBL_EPSILON)
00039     return xc;
00040
00041   double dx = c * ((A2 - A1) / sum);
00042   return xc + dx;
00043 }
```

5.36 discrsd.c

См. документацию.

```
00001
00013 #include "discrsd.h"
00014 #include <math.h>
00015 #include <float.h>
00016
00032 double discr2sd(double c, double A1, double A2,
00033                 double x1, double x2)
00034 {
00035   double xc = (x1 + x2) * 0.5;
00036   double sum = A2 + A1;
00037
00038   if (fabs(sum) < DBL_EPSILON)
00039     return xc;
00040
00041   double dx = c * ((A2 - A1) / sum);
00042   return xc + dx;
00043 }
```

Предметный указатель

_USE_MATH_DEFINES
    discr_common.c, 70

a
    ax_pair_t, 25
    axstr_t, 26
ax_pair_t, 25
    a, 25
    x, 25
axstr_t, 25
    a, 26
    x, 26

checkuse3d
    discr_common.c, 70
    discr_common.h, 36
cmp_by_x
    discr_auto.c, 62
compare_by_x
    discrea.c, 80
conv3to2
    discr_common.c, 70
    discr_common.h, 37

discr2
    discr_common.c, 71
    discr_common.h, 38
discr2cg
    discrcg.c, 77
    discrcg.h, 45
discr2sd
    discrsd.c, 87
    discrsd.h, 55
discr3
    discr_common.c, 72
    discr_common.h, 39
discr3_
    discr_common.c, 74
    discr_common.h, 41
discr3_auto
    discr_auto.c, 62
    discr_auto.h, 31
discr3_extrap_gauss
    discr_auto.c, 63
    discr_auto.h, 32
discr3cg
    discrcg.c, 78
    discrcg.h, 46
discr3ea
    discrea.c, 80
    discrea.h, 48
discr3eaY
    discrea.c, 82
    discrea.h, 50
discr3qa
    discrqa.c, 85
    discrqa.h, 53
discr5ea
    discr5ea.c, 57
    discr5ea.h, 28
discr5ea.c, 56, 60
    discr5ea, 57
    discr5qa, 58
    lsq5_peak, 59
discr5ea.h, 27, 30
    discr5ea, 28
    discr5qa, 29
discr5qa
    discr5ea.c, 58
    discr5ea.h, 29
discr_auto.c, 61, 66
    cmp_by_x, 62
    discr3_auto, 62
    discr3_extrap_gauss, 63
    discr_is_monotonic, 65
discr_auto.h, 30, 35
    discr3_auto, 31
    discr3_extrap_gauss, 32
    discr_is_monotonic, 34
discr_common.c, 69, 74
    _USE_MATH_DEFINES, 70
    checkuse3d, 70
    conv3to2, 70
    discr2, 71
    discr3, 72
    discr3_, 74
discr_common.h, 35, 42
    checkuse3d, 36
    conv3to2, 37
    discr2, 38
    discr3, 39
    discr3_, 41
discr_is_monotonic
    discr_auto.c, 65
    discr_auto.h, 34
discr_types.h, 43, 44
    DISCRTYPE, 44
    DT_CG, 44
    DT_EA, 44

DT_NO, 44
DT_QA, 44
DT_SD, 44
discrcg.c, 77, 79
discr2cg, 77
discr3cg, 78
discrncg, 78
discrcg.h, 44, 47
discr2cg, 45
discr3cg, 46
discrncg, 46
discrea.c, 79, 83
compare_by_x, 80
discr3ea, 80
discr3eaY, 82
discrea.h, 48, 52
discr3ea, 48
discr3eaY, 50
discrncg
discrcg.c, 78
discrcg.h, 46
discrqa.c, 84, 86
discr3qa, 85
discrqa.h, 52, 54
discr3qa, 53
discrsd.c, 86, 88
discr2sd, 87
discrsd.h, 54, 56
discr2sd, 55
DISCRTYPE
discr_types.h, 44
DT_CG
discr_types.h, 44
DT_EA
discr_types.h, 44
DT_NO
discr_types.h, 44
DT_QA
discr_types.h, 44
DT_SD
discr_types.h, 44

FFT-дискриминаторы частоты, 19
fft.md, 27
formulas.md, 27

lsq5_peak
discr5ea.c, 59

mainpage.md, 27
methods_visual.md, 27

recommendations.md, 27

tests.md, 27

x
ax_pair_t, 25
axstr_t, 26

Визуальный разбор методов, 8
Главная страница, 1
Итоговые рекомендации, 11
Математические формулы, 4
Тестирование, 16